

The CKA Deep Reference

A hands-on textbook for the Certified Kubernetes Administrator exam

Kubernetes • kubectl • Deep concepts + exam speed

Built from real cluster concepts & the full CKA curriculum
Includes 30 revision / LinkedIn posts

Contents

Part I — Foundations

01. Kubernetes Architecture

02. kubectl & Imperative Speed

03. Pods & Multi-Container Patterns

04. Namespaces

05. Labels, Selectors & Annotations

Part II — Workloads, Config, Networking & Security

06. ReplicaSets, Deployments & Rollouts

07. DaemonSets

08. Jobs & CronJobs

09. Scheduling: nodeSelector & Node Affinity

10. Taints & Tolerations

11. Requests, Limits & Resource Management

12. Autoscaling: HPA, VPA & Cluster Autoscaler

13. Health Probes: Liveness, Readiness & Startup

14. ConfigMaps & Secrets

15. Services: ClusterIP, NodePort & LoadBalancer

16. Ingress & the Gateway API

17. NetworkPolicies

18. CoreDNS & Service Discovery

19. Storage: Volumes, PV, PVC & StorageClasses

20. RBAC & ServiceAccounts

21. Security Contexts

22. Cluster PKI, TLS & Certificates

Part III — Cluster Administration & Troubleshooting

23. kubeadm Cluster Installation

24. Cluster Upgrades with kubeadm

25. etcd Backup & Restore

26. Static Pods

27. Node Maintenance: cordon, drain & uncordon

28. The Troubleshooting Workflow

29. Node & Container Debugging: crictl, journalctl & systemd

30. Helm

31. Kustomize

Part IV — 30 Posts for Revision & Sharing

32. 30 Posts for Revision & Sharing

PART I

Foundations

The mental model of a Kubernetes cluster, the `kubectl` muscle memory the exam rewards, and the primitives everything else is built on: Pods, Namespaces, and Labels.

Kubernetes Architecture

Kubernetes is a **declarative, control-loop driven** system: you tell it the desired state of the world, and a set of independent components continuously drive reality toward that state. Understanding which component owns which job — and where each one keeps its config — is the difference between guessing and diagnosing on the exam. A cluster splits into a **control plane** (the brain, usually on dedicated nodes) and **worker nodes** (where your Pods actually run). Every node also runs the node-level components.

Control plane components

Component	Job	Notes
`kube-apiserver`	Front door to the cluster. The only component that talks to etcd.	Stateless, horizontally scalable, exposes the REST API
`etcd`	Consistent key-value store — the single source of truth for all cluster state.	Raft consensus; back it up (see backup chapter)
`kube-scheduler`	Watches for unscheduled Pods and assigns each to a node.	Only sets `spec.nodeName`; does not start the Pod
`kube-controller-manager`	Runs the built-in controllers (Node, ReplicaSet, Deployment, Job, endpoints, service-account, etc.) as reconciliation loops.	One binary, many controllers
`cloud-controller-manager`	Bridges to a cloud provider (LoadBalancer services, node lifecycle, routes).	Absent on bare-metal / local clusters

The critical mental model: **the apiserver is the hub, and everything else is a spoke**. The scheduler, controller-manager, and kubelet never talk to etcd directly — they all watch and update objects through the apiserver. This is why `kubectl` can inspect everything: it is just an HTTPS client of the apiserver.

Node components

Component	Job
`kubelet`	The node agent. Watches the apiserver for Pods bound to its node, then calls the container runtime to run them. Reports node/pod status back. Runs static pods.
`kube-proxy`	Implements Service networking on each node (iptables/IPVS rules, or delegated to a CNI in some setups).
Container runtime (CRI)	Actually runs containers — containerd or CRI-O. Talks to the kubelet over the **Container Runtime Interface**. Docker/dockershim was removed in 1.24.

> 📌 **NOTE** The kubelet is the **only** control component that is not itself a Pod managed by the control plane — it is a systemd service on the node. It bootstraps everything else, including the static pods that form the control plane in kubeadm clusters.

The declarative reconciliation loop

Every controller runs the same loop: **observe** current state (via a watch on the apiserver), **compare** to the desired state (the object's `spec`), and **act** to close the gap, then record what it sees in `status`. There is no central orchestrator issuing commands — the system is *eventually consistent* and self-healing. Delete a Pod owned by a ReplicaSet and the ReplicaSet controller notices the count dropped below `spec.replicas` and creates a replacement. This is why you almost never `create` bare Pods in production: nothing reconciles them.

How a Pod gets scheduled and started, end to end

This exact sequence is a favourite exam-explanation and the key to debugging a stuck Pod:

1. **Submit:** `k apply` / `k run` sends the Pod object to the `kube-apiserver`, which validates it and (through admission) writes it to `etcd`. The Pod now exists with `status.phase: Pending` and **no** `nodeName`.
2. **Schedule:** the `kube-scheduler`, watching for Pods with an empty `nodeName`, filters feasible nodes (resources, taints, affinity, node selectors) and scores them, then writes the chosen node into `spec.nodeName` via the apiserver (a "binding").
3. **Sync:** the `kubelet` on that node, watching for Pods bound to it, sees the new Pod and begins the sync loop.
4. **Run:** the kubelet calls the container runtime over the **CRI** to pull images, set up the network (via CNI) and volumes, and start the containers.
5. **Report:** the kubelet updates `status` (container statuses, IP, phase → `Running`) back through the apiserver into etcd.

> 🎯 **EXAM TIP** Map failures to steps: stuck `Pending` with no node → scheduler problem (no feasible node / scheduler down). Node assigned but `ContainerCreating` → kubelet/CRI/image/volume problem — `describe pod` and check the node's kubelet.

Where the config lives

On a kubeadm cluster the control plane runs as **static pods**: the kubelet watches a directory and runs any manifest it finds there, independent of the apiserver.

Path	What's there
`/etc/kubernetes/manifests/`	Static pod manifests: `kube-apiserver.yaml`, `etcd.yaml`, `kube-scheduler.yaml`, `kube-controller-manager.yaml`. Edit a file here and the kubelet restarts that pod.
`/etc/kubernetes/*.conf`	Kubeconfig files for the components (`admin.conf`, `kubelet.conf`, etc.).
`/etc/kubernetes/pki/`	Cluster certificates and keys (CA, apiserver, etcd certs).
`/var/lib/kubelet/`	Kubelet working dir: `config.yaml` (kubelet's own config), pod volumes, plugin sockets.
`/var/lib/etcd/`	etcd data directory (what you snapshot for backups).
`/etc/kubernetes/manifests` path	Set by the kubelet's `staticPodPath` in `/var/lib/kubelet/config.yaml`.

> ⚠️ **GOTCHA** Because control-plane pods are **static pods**, you cannot fix a broken apiserver with `kubectl` (the API is down). SSH to the control-plane node and edit the file in `/etc/kubernetes/manifests/` — the kubelet

> reconciles it within seconds. A YAML typo there silently prevents the pod from starting; check `/var/log/` and `crictl ps`.

Inspecting components

```bash
# Control plane and node health
k get componentstatuses            # (deprecated but still on the exam)
k get pods -n kube-system          # the control-plane static pods live here
k get nodes -o wide

# Look at a static pod manifest / the kubelet config
sudo cat /etc/kubernetes/manifests/kube-apiserver.yaml
sudo cat /var/lib/kubelet/config.yaml

# When the API is UP, read component logs as pod logs
k logs -n kube-system kube-apiserver-controlplane
k logs -n kube-system kube-scheduler-controlplane

# When the API is DOWN, go under the kubelet + runtime
sudo systemctl status kubelet
sudo journalctl -u kubelet -f
sudo crictl ps                     # containers the runtime is actually running
sudo crictl logs <container-id>
```

Exam tips

- Know the one-liner for each component's job cold — the exam tests *which* component you'd inspect for a given symptom.
- Control-plane pods are **static pods** defined in `/etc/kubernetes/manifests/`. To restart one, move its manifest out of the directory and back in.
- The **scheduler only sets** `nodeName`; the **kubelet actually starts** the Pod. A Pod with no `nodeName` that stays Pending is a scheduling problem.
- If `kubectl` itself fails to connect, the apiserver or your kubeconfig is the suspect — fall back to `systemctl`, `journalctl -u kubelet`, and `crictl`.
- Only the **apiserver** talks to **etcd**. Corrupting or losing etcd loses all cluster state — hence the backup chapter.

kubectl & Imperative Speed

The CKA is a **hands-on, time-boxed** exam: ~15-20 tasks in 2 hours, graded on the resulting cluster state, not on how you got there. Speed is a first-class skill. The fastest candidates rarely hand-write YAML from scratch — they **generate** it imperatively and edit only the delta. This chapter is about the muscle memory that saves minutes per task.

Imperative vs declarative

Imperative commands tell the cluster to do a thing now (`k run`, `k create`, `k expose`, `k scale`). **Declarative** means you write a manifest describing desired state and `k apply` it — Kubernetes figures out the diff. On the exam: use imperative to *create fast*, use `--dry-run=client -o yaml` to *generate a manifest* you then tweak, and use `apply` for anything you'll modify repeatedly.

The setup that pays for itself in minute one

Do this at the very start of the exam in your terminal. It's already partly configured on the exam VM, but verify it:

```bash
alias k=kubectl
export do="--dry-run=client -o yaml"    # "dry output"
export now="--force --grace-period=0"    # instant delete

# Now creating a manifest is trivial:
k run nginx --image=nginx $do > pod.yaml
k create deploy web --image=nginx $do > deploy.yaml
k delete pod nginx $now                  # skip the 30s graceful wait
```

> 🎯 **EXAM TIP** The `$do` / `$now` pattern is the single biggest time-saver on the exam. `k ... $do > file.yaml` gives you a valid, correctly indented skeleton in one keystroke burst — never start from a blank file.

Autocompletion & the k alias

```bash
source <(kubectl completion bash)
alias k=kubectl
complete -o default -F __start_kubectl k   # make TAB completion work for 'k' too
```

Tab-completion of resource names, namespaces, and flags eliminates typos on long pod names.

Generating YAML fast

```bash
# Pods, deployments, jobs, cronjobs — all support $do
k run pod1 --image=nginx --port=80 $do
k run tmp --image=busybox --restart=Never -it --rm -- sh   # throwaway debug pod
k create deploy web --image=nginx --replicas=3 $do
k create job pi --image=perl -- perl -e 'print 42' $do
k create cronjob cj --image=busybox --schedule="*/1 * * * *" -- date $do
k create cm app-cfg --from-literal=KEY=val $do
```

```
k create secret generic s1 --from-literal=pw=1234 $do
k expose deploy web --port=80 --target-port=8080 $do   # generates a Service
```

📌 **NOTE** `k run` creates a Pod; `k create deploy` creates a Deployment. Not every field is exposable via flags — generate the skeleton, then `k edit` or hand-add the missing bits (volumes, probes, resources).

kubectl explain — the built-in field reference

You cannot browse the web in the exam, but `explain` is your live API docs.

BASH

```
k explain pod.spec.containers
k explain pod.spec.containers.livenessProbe --recursive
k explain deploy.spec.strategy
k api-resources                    # every kind, its shortname, apiVersion, namespaced?
k api-versions
```

Selecting & filtering output

BASH

```
# Field selectors (server-side filter on specific fields)
k get pods --field-selector status.phase=Running
k get pods --field-selector spec.nodeName=node01
k get events --field-selector type=Warning

# Label selectors
k get pods -l app=web,tier≠cache --show-labels

# jsonpath — extract exactly what you need
k get pods -o jsonpath='{.items[*].metadata.name}'
k get nodes -o jsonpath='{.items[*].status.addresses[?(@.type=="InternalIP")].address}'
k get pods -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName
k get pods --sort-by=.metadata.creationTimestamp
```

apply vs create vs replace vs edit

Command	Behaviour	Use when
`k create -f`	Creates; **errors if it already exists**.	One-shot creation
`k apply -f`	Creates or updates via a 3-way merge; tracks the last-applied config.	Anything you'll edit again; GitOps
`k replace -f`	Deletes & recreates the object's spec; **errors if it doesn't exist**.	Full overwrite of an existing object
`k replace --force -f`	Deletes and recreates the object entirely.	Immutable-field changes (e.g. a Pod)
`k edit`	Opens the live object in `$EDITOR`, applies on save.	Quick in-place field change

⚠️ **GOTCHA** Mixing `create` and `apply` on the same object can cause "conflict" warnings because `create` doesn't set the last-applied annotation. Pick `apply` and stick with it for objects you'll revisit. Also: many Pod fields

> are **immutable** — `k edit` will reject the change and you'll need `k replace --force` or delete/recreate.

Contexts & namespaces

The exam has multiple clusters. Each task tells you which context to switch to — **do it first, every time**, or you'll grade against the wrong cluster.

```bash
k config get-contexts
k config current-context
k config use-context cluster-2      # ALWAYS run the context line from the task

# Pin a default namespace so you stop typing -n
k config set-context --current --namespace=dev
k config view --minify | grep namespace   # confirm
```

The community tools `kubens` / `kubectx` do the same thing faster (`kubens dev` to switch namespace, `kubectx cluster-2` for context) but aren't guaranteed installed — know the raw `k config` commands.

> 🎯 **EXAM TIP** Setting the namespace with `k config set-context --current --namespace=X` means every subsequent command targets `X` . This is faster and less error-prone than appending `-n X` to twenty commands in a namespaced task.

vim setup for the exam

YAML is whitespace-sensitive and tabs break it. Put this in `~/.vimrc` at the start of the exam:

```bash
cat <<'EOF' > ~/.vimrc
set expandtab      " tabs become spaces
set tabstop=2
set shiftwidth=2
set number
EOF
```

In vim, `:set paste` before pasting blocks (prevents auto-indent mangling), and `gg=G` re-indents. But prefer generating YAML with `$do` so you edit less by hand.

Exam tips

- First action on **every** task: run the provided `kubectl config use-context` line.
- `k run/create ... $do > f.yaml` then edit — never write manifests blank.
- Use `k explain --recursive` when you forget a field path; it beats guessing.
- `k delete ... $now` to skip graceful termination when a task just needs the object gone.
- Set the default namespace per task instead of repeating `-n` .
- `-o jsonpath` and `--field-selector` answer "list only the X that match Y" questions in one line — great for tasks that ask you to write results to a file.
- If `k edit` rejects your change, the field is immutable — use `k replace --force -f` .

Pods & Multi-Container Patterns

A **Pod** is the smallest deployable unit in Kubernetes — not a container, but a *wrapper around one or more containers* that share a network identity and storage. You almost never create Pods directly in production (a Deployment or Job manages them), but the Pod is the atom everything else is built from, and the exam expects you to know its anatomy, lifecycle, and the multi-container patterns cold.

What a Pod actually is

Containers in a Pod are **co-scheduled on one node** and share two things:

- **Network namespace:** one IP address, one port space. Containers reach each other over `localhost` and cannot both bind the same port.
- **Storage:** any `volume` declared in the Pod can be mounted into multiple containers, so they exchange files.

They do **not** share a filesystem root or PID namespace by default (PID sharing is opt-in via `shareProcessNamespace: true`). Under the hood a hidden **pause container** holds the network namespace so app containers can come and go without losing the Pod IP.

> 📌 **NOTE** Because containers in a Pod share an IP, a sidecar proxy on `localhost:15001` can intercept the app's traffic with zero network config in the app — this is exactly how service meshes work.

Pod lifecycle & phases

Phase	Meaning
`Pending`	Accepted but not all containers running — being scheduled, or images pulling, or init containers running.
`Running`	Bound to a node, at least one container running.
`Succeeded`	All containers exited 0 and won't restart.
`Failed`	All containers terminated, at least one non-zero.
`Unknown`	Node unreachable — kubelet not reporting.

Container-level states you'll see in `describe` / `get pod`: `Waiting` (with a reason like `ContainerCreating`, `ImagePullBackOff`, `CrashLoopBackOff`), `Running`, and `Terminated`.

Restart policy

`spec.restartPolicy` governs restarts of containers *within* the Pod:

Value	Behaviour	Used by
`Always` (default)	Restart containers whenever they exit.	Deployments, long-running apps
`OnFailure`	Restart only on non-zero exit.	Jobs that should retry
`Never`	Never restart.	One-shot / Jobs

> ⚠️ **GOTCHA** Restart policy applies to the whole Pod, not per-container, and it restarts containers **in place** — it does not reschedule the Pod. A Pod that keeps crashing shows `CrashLoopBackOff`: the kubelet is backing off (10s, 20s, 40s… up to 5m) between restart attempts. `k logs --previous` shows the last crashed container's output.

A basic Pod manifest

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web
  labels:
    app: web
spec:
  restartPolicy: Always
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
      resources:
        requests: {cpu: "100m", memory: "128Mi"}
        limits:   {cpu: "250m", memory: "256Mi"}
```

Init containers

Init containers run **to completion, one after another, before** the app containers start. They're perfect for setup: waiting on a dependency, cloning config, running a migration. If an init container fails, the kubelet restarts it (subject to `restartPolicy`) and the app containers never start.

```yaml
apiVersion: v1
kind: Pod
metadata: {name: app}
spec:
  initContainers:
    - name: wait-for-db
      image: busybox
      command: ["sh","-c","until nc -z db 5432; do echo waiting; sleep 2; done"]
  containers:
    - name: app
      image: myapp:1.0
```

The sidecar pattern (and native sidecars)

A **sidecar** is a helper container that runs alongside the main app for its whole life — log shippers, proxies, config reloaders. Classically it's just a second entry in `containers:`. Since 1.28/1.29, Kubernetes has **native sidecars**: an init container with `restartPolicy: Always`, which starts before the main containers and keeps running — solving the old problem of sidecars not being ready in time or blocking Job completion.

```yaml
spec:
  initContainers:
```

```
    - name: log-shipper          # native sidecar
      image: fluent/fluent-bit
      restartPolicy: Always      # this is what makes it a sidecar, not a normal init
      volumeMounts:
        - {name: logs, mountPath: /var/log/app}
  containers:
    - name: app
      image: myapp:1.0
      volumeMounts:
        - {name: logs, mountPath: /var/log/app}
  volumes:
    - name: logs
      emptyDir: {}
```

Sharing localhost and a volume

Two classic proofs of Pod-sharing: a container curling its neighbour over `localhost`, and two containers exchanging a file through an `emptyDir`.

YAML

```yaml
apiVersion: v1
kind: Pod
metadata: {name: shared}
spec:
  containers:
    - name: writer
      image: busybox
      command: ["sh","-c","echo hi > /data/msg; sleep 3600"]
      volumeMounts: [{name: scratch, mountPath: /data}]
    - name: reader
      image: busybox
      command: ["sh","-c","cat /data/msg; sleep 3600"]
      volumeMounts: [{name: scratch, mountPath: /data}]
  volumes:
    - name: scratch
      emptyDir: {}
```

The three multi-container patterns

Pattern	What the helper does	Example
Sidecar	Augments/extends the main container.	Log tailer, cert reloader, mesh proxy
Ambassador	Proxies the app's *outbound* connections to the outside world, so the app just talks to `localhost`.	Local proxy to a sharded DB or remote API
Adapter	Transforms the app's output into a standard format for external consumers.	Reformatting logs/metrics for Prometheus

🎯 **EXAM TIP** Memory hook: **Ambassador = outbound** (represents the app to the world), **Adapter = normalizes output** (translates for the world), **Sidecar = general helper**. All three rely on the shared network + volume of the Pod.

Essential commands

```bash
k run web --image=nginx $do > pod.yaml   # generate a skeleton ($do = --dry-run=client -o yaml)
k get pod web -o wide                    # NODE and IP
k describe pod web                       # Events, container states, restart count
k logs web -c nginx                      # -c chooses the container in a multi-container Pod
k logs web -c app --previous             # last crashed instance
k exec -it web -c nginx -- sh            # shell into a specific container
k get pod web -o jsonpath='{.status.phase}'
```

Exam tips

- In a multi-container Pod, `k logs` and `k exec` **require** `-c <container>` or they error / pick the first container.
- `CrashLoopBackOff` → `k logs --previous`. `ImagePullBackOff` → wrong image name/tag or missing pull secret (`describe` shows the exact error).
- Init containers run **sequentially to completion** before app containers; a stuck init keeps the Pod `Pending` / `Init:0/1`.
- A **native sidecar** = init container with `restartPolicy: Always`. Know it exists in 1.29+.
- Containers share `localhost` and volumes but not filesystem roots — the shared data path must be an explicit `emptyDir` (or other volume) mounted in both.

Namespaces

Namespaces carve a single physical cluster into multiple virtual clusters — a way to group and isolate resources for different teams, environments, or applications. They provide a **naming scope** and a boundary for quotas and policies, but they are *not* a security boundary by themselves. Knowing exactly what a namespace isolates — and what it doesn't — is a recurring exam theme.

What namespaces isolate (and what they don't)

Isolated by namespace	NOT isolated
Object names (a `web` Pod can exist in each namespace)	Nodes — cluster-scoped, shared
ResourceQuota & LimitRange scope	PersistentVolumes, StorageClasses — cluster-scoped
RBAC Role bindings (namespaced)	Network traffic — Pods can reach other namespaces unless a NetworkPolicy blocks it
Most workload objects (Pods, Deployments, Services, ConfigMaps, Secrets, PVCs)	ClusterRoles, Nodes, PVs, StorageClasses, Namespaces themselves

> ⚠️ **GOTCHA** A namespace is **not** a network or security wall on its own. By default, a Pod in `dev` can open a connection to a Pod in `prod`. Real isolation needs **NetworkPolicies** (traffic) and **RBAC** (API access). Do not assume "different namespace" means "can't talk".

Quick check of which resources are namespaced:

```bash
k api-resources --namespaced=true    # namespaced kinds
k api-resources --namespaced=false   # cluster-scoped kinds (nodes, pv, sc, ns ... )
```

The default namespaces

Namespace	Purpose
`default`	Where objects land if you don't specify one.
`kube-system`	Control-plane and add-on components (CoreDNS, kube-proxy, control-plane static pods).
`kube-public`	World-readable; holds cluster info. Rarely used directly.
`kube-node-lease`	Holds node heartbeat Lease objects for faster node-failure detection.

Creating and using namespaces

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: dev
```

```bash
k create namespace dev
k get ns
k get pods -n dev                    # target a namespace
k get pods -A                        # --all-namespaces
k run web --image=nginx -n dev

# Stop typing -n: pin the namespace in your context
k config set-context --current --namespace=dev
k config view --minify | grep namespace
```

> 🎯 **EXAM TIP** On namespaced exam tasks, set the default namespace once with `k config set-context --current --namespace=X` instead of appending `-n X` to every command. Fewer keystrokes, fewer "created in the wrong namespace" mistakes.

DNS across namespaces

CoreDNS gives every Service a DNS name. Within the **same** namespace a Pod can reach a Service by its short name (`web`). Across namespaces you must qualify it. The full form is:

```bash
<service>.<namespace>.svc.cluster.local

# From a Pod in 'default', reaching the 'db' service in 'dev':
curl http://db.dev.svc.cluster.local
curl http://db.dev                   # short form also resolves (search domains)
```

Pods themselves get DNS records too (for headless services), in the form `<pod-ip-dashed>.<namespace>.pod.cluster.local` . The `svc.cluster.local` suffix is the cluster domain, configurable but almost always this default.

> 📌 **NOTE** The `<service>.<namespace>` segment is why namespaces matter for connectivity: a Service named `web` in `dev` and one in `prod` are distinct DNS names (`web.dev` vs `web.prod`) and never collide.

ResourceQuota — cap total consumption per namespace

A **ResourceQuota** limits the aggregate resources a namespace may consume: total CPU/memory requests and limits, and object counts (how many Pods, Services, PVCs, etc.).

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: dev
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
```

```
    pods: "20"
    persistentvolumeclaims: "5"
```

⚠️ **GOTCHA** Once a ResourceQuota that constrains CPU/memory exists in a namespace, every Pod created there **must specify** the corresponding requests/limits — otherwise the Pod is rejected with a "must specify limits/requests" error. Pair a quota with a LimitRange so Pods get defaults automatically.

LimitRange — defaults & bounds per object

A **LimitRange** operates on individual objects in a namespace: it sets **default** requests/limits for containers that omit them, and enforces **min/max** bounds.

YAML

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: dev-limits
  namespace: dev
spec:
  limits:
    - type: Container
      default:                # applied as limits if none set
        cpu: 500m
        memory: 256Mi
      defaultRequest:         # applied as requests if none set
        cpu: 100m
        memory: 128Mi
      max:  {cpu: "1",  memory: 512Mi}
      min:  {cpu: 50m,  memory: 64Mi}
```

	ResourceQuota	LimitRange
Scope	Whole namespace (aggregate)	Per container/Pod/PVC
Enforces	Total consumption & object counts	Defaults + min/max per object
Failure	Rejects create when the sum would exceed the cap	Rejects create when an object violates min/max

Essential commands

BASH

```bash
k create ns dev
k get resourcequota -n dev
k describe resourcequota dev-quota -n dev   # shows Used vs Hard
k get limitrange -n dev
k describe limitrange dev-limits -n dev
k delete ns dev                             # deletes EVERYTHING namespaced inside it
```

⚠️ **GOTCHA** `k delete ns dev` cascades — it removes **all** namespaced objects in that namespace. A namespace stuck in `Terminating` usually has a finalizer or an orphaned API resource; check `k get ns dev -o yaml` under `status`.

Exam tips

- Namespaces scope names, quotas, and RBAC — **not** network or PVs. Nodes, PVs, and StorageClasses are cluster-scoped.
- Cross-namespace access is `service.namespace.svc.cluster.local` (or just `service.namespace`). Same-namespace = short name.
- **ResourceQuota** = aggregate ceiling per namespace; **LimitRange** = per-object defaults and bounds. They're often tested together.
- If Pods start getting rejected for "must specify limits", a ResourceQuota is in play — add a LimitRange to supply defaults.
- Set your default namespace with `k config set-context --current --namespace=X` to save time on namespaced tasks.

Labels, Selectors & Annotations

Labels are the glue of Kubernetes. They are key/value pairs attached to objects that let controllers and services find the right set of Pods without hard-coding names. Nearly every "how does X know which Pods to manage?" question resolves to a **label selector**. **Annotations** look similar but serve the opposite purpose — non-identifying metadata for tools and humans. Getting the distinction and the selector syntax right is quick, high-value exam points.

Labels vs annotations

	Labels	Annotations
Purpose	Identify & select objects	Attach arbitrary, non-identifying metadata
Selectable?	Yes — the basis of selectors	No — you cannot select on them
Value size	Short (≤63 chars, restricted charset)	Large, structured data allowed (URLs, JSON, config)
Typical use	`app`, `tier`, `env`, `release`	build info, tool config, change-cause, checksums

```yaml
metadata:
  labels:
    app: web
    tier: frontend
    env: prod
  annotations:
    kubernetes.io/change-cause: "roll out v2"
    example.com/config: '{"retries": 3}'
```

> 📌 **NOTE** Rule of thumb: if something needs to **find** the object, it's a label. If it's just **information about** the object (that a controller or human reads, but never selects on), it's an annotation.

Equality vs set-based selectors

Equality-based selectors match exact values with `=`, `==`, or `≠`. **Set-based** selectors match membership with `in`, `notin`, and `exists`. On the command line you combine them with `-l`; comma means AND.

```bash
# Equality
k get pods -l app=web
k get pods -l 'tier≠cache'
k get pods -l app=web,env=prod        # AND

# Set-based
k get pods -l 'env in (prod,staging)'
k get pods -l 'tier notin (cache)'
k get pods -l 'app'                    # exists: has the key 'app' at all
k get pods -l '!app'                   # does not have the key 'app'
```

How controllers use selectors

Deployments, ReplicaSets, and Services don't own Pods by name — they own whatever Pods match their **selector**. This indirection is what enables self-healing and rolling updates: the controller keeps counting Pods that match and reconciles.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: {name: web}
spec:
  replicas: 3
  selector:
    matchLabels:            # which Pods this Deployment owns
      app: web
  template:
    metadata:
      labels:
        app: web            # MUST match the selector, or creation fails
    spec:
      containers:
        - name: nginx
          image: nginx
```

⚠️ **GOTCHA** The Deployment/ReplicaSet `spec.selector.matchLabels` **must** match the Pod template's `metadata.labels`, or the API rejects it with `selector does not match template labels`. And a Deployment's `selector` is **immutable** after creation — you cannot edit it; you must recreate the object.

matchLabels vs matchExpressions

Modern selectors (Deployments, ReplicaSets, NetworkPolicies, affinity) support two forms, which combine with AND:

```yaml
selector:
  matchLabels:
    app: web              # equality (implicit AND across entries)
  matchExpressions:
    - key: tier
      operator: In        # In | NotIn | Exists | DoesNotExist
      values: [frontend, api]
    - key: legacy
      operator: DoesNotExist   # no 'values' for Exists/DoesNotExist
```

`matchLabels` is the simple equality shorthand; `matchExpressions` is the set-based, more expressive form. All conditions across both must be satisfied.

Services select Pods the same way

A Service builds its Endpoints from Pods matching its selector. The old `Service.spec.selector` uses **only** equality-style map (no matchExpressions):

```yaml
apiVersion: v1
kind: Service
metadata: {name: web}
```

```
spec:
  selector:
    app: web              # equality map only for core Service
  ports:
    - port: 80
      targetPort: 80
```

🎯 **EXAM TIP** If a Service has **no endpoints**, the number-one cause is a selector that doesn't match any Pod's labels. Check with `k get pods --show-labels` and `k get endpoints <svc>` — empty endpoints means the selector missed.

Managing labels with kubectl

BASH

```
# Add / change / remove
k label pod web tier=frontend
k label pod web tier=backend --overwrite    # change an existing label
k label pod web tier-                        # trailing minus removes the label
k label pods --all env=prod                  # label everything

# Inspect
k get pods --show-labels
k get pods -L app -L tier                    # show chosen labels as columns
k get pods -l 'env in (prod)' --show-labels

# Annotate
k annotate pod web description="frontend web server"
k annotate pod web description-               # remove
```

Selectors on other commands

BASH

```
k delete pods -l app=web            # delete all matching Pods
k get all -l app=web               # all resource types with this label
k rollout restart deploy -l tier=frontend
```

Exam tips

- **Labels are selectable, annotations are not.** If a task says "select/find/target Pods by X", X must be a label.
- A Deployment/ReplicaSet fails to create if `selector.matchLabels` ≠ template labels. The selector is **immutable** afterwards.
- Service with no endpoints → selector mismatch. Verify with `--show-labels` and `k get endpoints`.
- Remove a label with a trailing minus: `k label pod web key-`. Overwrite needs `--overwrite`.
- `matchLabels` = equality shorthand; `matchExpressions` = set-based (`In/NotIn/Exists/DoesNotExist`). Core Services support equality maps only.
- Comma in `-l` means AND: `-l app=web,env=prod`.

PART II

Workloads, Config, Networking & Security

How you actually run and expose applications: controllers and rollouts, scheduling, resource governance, config & secrets, the full networking stack, storage, and the RBAC / TLS security model.

ReplicaSets, Deployments & Rollouts

A bare Pod is a single point of failure — if it dies, nothing brings it back. The controllers in this chapter add the two properties production workloads demand: **replication** (run N identical Pods and keep them running) and **controlled change** (roll out a new version without downtime, and roll back when it breaks). Deployments are the workhorse of the Workloads domain and one of the most heavily tested topics on the CKA — expect to create, scale, update, inspect, and roll back one under time pressure.

The controller hierarchy

You almost never create a ReplicaSet directly. A **Deployment** manages **ReplicaSets**, and each ReplicaSet manages **Pods**. Every time you change the Pod template, the Deployment creates a *new* ReplicaSet and gradually shifts replicas from the old one to the new one — that shift is the "rollout".

Object	Job	Namespaced
Pod	Runs containers	Yes
ReplicaSet	Keeps N Pods matching a selector alive	Yes
Deployment	Manages ReplicaSets to enable rollouts/rollbacks	Yes

ReplicaSet — the replication engine

A ReplicaSet has three parts: `replicas` (desired count), `selector` (which Pods it owns), and `template` (the Pod spec it stamps out). The controller runs a reconcile loop: count Pods matching the selector, then create or delete Pods until the count equals `replicas`. This is what gives you **self-healing** — delete a Pod and a replacement appears within seconds.

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: web-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web              # MUST match template labels below
  template:
    metadata:
      labels:
        app: web            # Pods get this label; selector finds them
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
```

⚠️ **GOTCHA** The `selector` is immutable and must match the `template.metadata.labels`. A ReplicaSet will **adopt** any pre-existing bare Pod that happens to match its selector — a classic gotcha when labels overlap.

Deployment — declarative updates

A Deployment wraps a ReplicaSet template and adds a `strategy` for how to replace Pods when the template changes. This is the object you should reach for by default.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels: {app: web}
spec:
  replicas: 3
  revisionHistoryLimit: 10      # how many old ReplicaSets to keep (default 10)
  selector:
    matchLabels: {app: web}
  strategy:
    type: RollingUpdate         # RollingUpdate (default) | Recreate
    rollingUpdate:
      maxSurge: 25%             # extra Pods allowed above replicas during rollout
      maxUnavailable: 25%       # Pods allowed to be down during rollout
  template:
    metadata:
      labels: {app: web}
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
```

Rolling update strategy — maxSurge and maxUnavailable

During a RollingUpdate the Deployment scales the new ReplicaSet up and the old one down in lockstep, bounded by two knobs. Both accept an absolute number or a percentage (rounded against `replicas`):

- **maxSurge** — how many Pods may exist *above* the desired count. Higher = faster rollout, more peak capacity used. `maxSurge` rounds **up**.
- **maxUnavailable** — how many Pods may be *missing* below desired count. Higher = faster but less headroom. `maxUnavailable` rounds **down**.

> 📌 **NOTE** You cannot set both to 0 — the rollout would have no way to make progress. Setting `maxUnavailable: 0` gives a strict "surge then remove" rollout that never drops below full capacity.

Recreate strategy

`strategy.type: Recreate` terminates **all** old Pods before creating any new ones. It causes downtime, so it is only for apps that cannot run two versions at once (e.g. an exclusive lock on a shared volume or an incompatible schema).

Managing rollouts

Any change to the `template` (image, env, resources, labels) triggers a new rollout. Changing only `replicas` does **not** — that is a scale, not a revision, and it does not create a new ReplicaSet.

```bash
# Create imperatively, then export YAML
k create deploy web --image=nginx:1.25 --replicas=3
k create deploy web --image=nginx:1.25 --dry-run=client -o yaml > web.yaml

# Update the image (records a new revision)
k set image deploy/web nginx=nginx:1.26
k set image deploy/web nginx=nginx:1.26 --record   # annotate the change-cause

# Watch the rollout
k rollout status deploy/web
k rollout history deploy/web            # list revisions
k rollout history deploy/web --revision=2

# Roll back
k rollout undo deploy/web               # to previous revision
k rollout undo deploy/web --to-revision=1

# Pause / resume (batch several edits into one rollout)
k rollout pause deploy/web
k set resources deploy/web -c nginx --limits=cpu=200m
k rollout resume deploy/web

# Restart (re-roll all Pods, e.g. to pick up a changed Secret)
k rollout restart deploy/web
```

Scaling & self-healing

```bash
k scale deploy/web --replicas=5
k scale deploy/web --replicas=5 --current-replicas=3   # conditional (safe) scale
k get rs -l app=web                     # old RS at 0, current RS at desired count
k get deploy web -o wide                # READY / UP-TO-DATE / AVAILABLE columns
```

Delete a Pod owned by the Deployment and the ReplicaSet recreates it immediately — the `ownerReferences` on each Pod point back to its ReplicaSet, and the reconcile loop restores desired state.

> 🎯 **EXAM TIP** `revisionHistoryLimit` controls how many **old, scaled-to-zero** ReplicaSets are retained for rollback. If `rollout undo` says "no rollout history found", the old ReplicaSet was pruned because the limit was too low.

Reading the status columns

Column	Meaning
READY	Ready Pods / desired (e.g. `3/3`)
UP-TO-DATE	Pods running the latest template
AVAILABLE	Pods that have passed `minReadySeconds` and readiness

Exam-day checklist

- Rollout stuck? → `k rollout status` then `k describe deploy` and `k get rs`. A bad image leaves new Pods in `ImagePullBackOff` while the old RS stays up — that is rolling update protecting you.
- Only template changes create revisions; `k scale` never does.
- `k set image`, `k scale`, and `k rollout` are the fastest imperative paths — memorise them.
- Selector and template labels must match, and the selector is immutable after creation.
- Use `k rollout undo --to-revision=N` with the number from `rollout history`, not the ReplicaSet name.

DaemonSets

A **DaemonSet** ensures that a copy of a Pod runs on **every node** (or every node matching a selector). Where a Deployment answers "run N replicas somewhere in the cluster", a DaemonSet answers "run exactly one replica on each node". This is the correct controller for anything that must be present per-node: log shippers, metrics agents, storage daemons, and CNI network plugins. It appears on the CKA both as a thing to create and as the reason certain system Pods land on control-plane nodes.

One-pod-per-node semantics

The DaemonSet controller does not pick a replica count — it watches the set of nodes. When a node joins the cluster, a Pod is added automatically; when a node is removed, its Pod is garbage-collected. There is no `replicas` field. Scheduling is handled by the default scheduler using node affinity that the controller injects per node, plus tolerations you supply.

Use case	Typical DaemonSet
Log collection	Fluent Bit, Fluentd, Filebeat
Node metrics	node-exporter, cAdvisor
Cluster networking (CNI)	Calico, Cilium, kube-proxy, flannel
Storage	CSI node plugins, Ceph, GlusterFS daemons

DaemonSet YAML

The spec mirrors a Deployment minus `replicas`: a `selector` and a Pod `template` whose labels must match the selector.

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: log-agent
  namespace: logging
  labels: {app: log-agent}
spec:
  selector:
    matchLabels: {app: log-agent}     # must match template labels
  updateStrategy:
    type: RollingUpdate               # RollingUpdate (default) | OnDelete
    rollingUpdate:
      maxUnavailable: 1               # how many node-Pods down at once
  template:
    metadata:
      labels: {app: log-agent}
    spec:
      tolerations:
        - key: node-role.kubernetes.io/control-plane
          operator: Exists
          effect: NoSchedule          # so it also lands on control-plane nodes
      containers:
```

```
      - name: fluent-bit
        image: fluent/fluent-bit:2.2
        volumeMounts:
          - name: varlog
            mountPath: /var/log
            readOnly: true
    volumes:
      - name: varlog
        hostPath:
          path: /var/log            # read the node's own logs
```

Tolerating control-plane taints

Control-plane nodes carry the taint `node-role.kubernetes.io/control-plane:NoSchedule`, which repels ordinary Pods. A DaemonSet that must cover *every* node (a CNI or metrics agent) needs a matching **toleration**, as shown above. Without it, the DaemonSet simply skips control-plane nodes and you get one fewer Pod than nodes.

> 📌 **NOTE** The scheduler also auto-adds tolerations for node conditions like `node.kubernetes.io/disk-pressure` and `unschedulable` to DaemonSet Pods so agents keep running even on cordoned or pressured nodes.

Targeting a subset of nodes

To run on only some nodes, add a `nodeSelector` or node affinity to the Pod template. The DaemonSet then places one Pod on each *matching* node only.

YAML

```
  spec:
    nodeSelector:
      disktype: ssd          # only nodes labelled disktype=ssd get a Pod
```

Update strategies

Strategy	Behaviour
`RollingUpdate` (default)	After a template change, delete and recreate node-Pods gradually, bounded by `maxUnavailable` (and optionally `maxSurge`, 1.22+).
`OnDelete`	New template applies only when you manually delete each old Pod — full control, useful for sensitive node agents.

> 🎯 **EXAM TIP** DaemonSets have `maxUnavailable` but historically no `maxSurge`, because you cannot have two of the same DaemonSet Pod on one node by default. `maxSurge` for DaemonSets is available from 1.22+ when you can tolerate a brief second Pod per node.

Essential commands

BASH

```
# There is no `kubectl create daemonset`; generate from a deployment and edit,
# or apply YAML directly.
```

```
k create deploy tmp --image=fluent/fluent-bit:2.2 --dry-run=client -o yaml > ds.yaml
# then change kind: Deployment → DaemonSet and remove replicas/strategy

k apply -f ds.yaml
k get ds -A                         # DESIRED CURRENT READY UP-TO-DATE AVAILABLE
k get ds log-agent -o wide
k describe ds log-agent             # events explain why a node has no Pod

# Roll and inspect
k rollout status ds/log-agent
k rollout history ds/log-agent
k rollout undo ds/log-agent

# See which node each Pod is on
k get pods -l app=log-agent -o wide
```

Reading the status columns

`k get ds` shows DESIRED (= number of nodes that should run it), CURRENT, READY, UP-TO-DATE, and AVAILABLE. If DESIRED is lower than your node count, a `nodeSelector`/affinity is filtering nodes, or an untolerated taint is repelling the Pod.

> ⚠️ **GOTCHA** A common exam trap: a DaemonSet shows fewer Pods than nodes and you are asked why. The usual answer is a control-plane taint with no matching toleration, or a `nodeSelector` that excludes some nodes. Check `k describe node <node>` Taints and the Pod template.

Exam-day checklist

- No `replicas` field — count is driven by nodes. Do not add one.
- To cover control-plane nodes, add a toleration for `node-role.kubernetes.io/control-plane:NoSchedule`.
- Use `nodeSelector`/affinity to restrict to a node subset.
- Default update strategy is `RollingUpdate`; `OnDelete` requires you to delete Pods to trigger the update.
- There is no imperative `create daemonset` — start from a Deployment YAML and change the kind, or write it by hand.

Jobs & CronJobs

Deployments and DaemonSets run Pods that are meant to stay up forever. A **Job** runs Pods to **completion** — a batch task that starts, does work, and exits successfully. A **CronJob** is a Job factory: it creates a Job on a repeating schedule. Together they cover backups, migrations, report generation, and any run-once-or-periodically workload. Expect the CKA to have you create a Job with a completion/parallelism/backoff requirement and a CronJob with a specific schedule.

How a Job tracks completion

A Job creates Pods and watches for them to terminate **successfully** (exit 0). It keeps creating replacements for failed Pods until it reaches the desired number of successes or exhausts its retry budget. The controlling fields:

Field	Meaning
`completions`	How many Pods must succeed for the Job to finish (default 1)
`parallelism`	How many Pods may run at once (default 1)
`backoffLimit`	Retries before the Job is marked Failed (default 6)
`activeDeadlineSeconds`	Wall-clock cap; Job is failed and Pods killed when exceeded
`ttlSecondsAfterFinished`	Auto-delete the finished Job (and its Pods) after N seconds

Job YAML

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: pi
spec:
  completions: 4          # need 4 successful Pods
  parallelism: 2          # run 2 at a time
  backoffLimit: 4         # up to 4 retries on failure
  activeDeadlineSeconds: 300
  template:
    spec:
      restartPolicy: Never      # MUST be Never or OnFailure (not Always)
      containers:
        - name: pi
          image: perl:5.34
          command: ["perl","-Mbignum=bpi","-wle","print bpi(2000)"]
```

⚠️ **GOTCHA** A Job's Pod template must set `restartPolicy: Never` or `OnFailure`. `Always` is rejected — a Job is not meant to run forever, so an always-restarting container would never let it complete.

restartPolicy: Never vs OnFailure

- **Never** — on failure the container is *not* restarted in place; the Job creates a brand-new Pod. You accumulate one Pod object per attempt (good for inspecting each failure's logs).

- **OnFailure** — the kubelet restarts the failed container in the *same* Pod. You see one Pod with a rising restart count instead of many Pods.

In both cases `backoffLimit` caps total failures before the Job gives up, with an exponential back-off between retries (capped at 6 minutes).

Parallel patterns

Pattern	Fields
Single run	`completions:1, parallelism:1`
Fixed completion count	`completions:N, parallelism:M`
Work queue	`parallelism:M`, no `completions` (Pods coordinate externally; any one succeeding drains the queue)

CronJob — scheduling Jobs

A CronJob wraps a Job template in a standard cron `schedule`. At each firing it creates one Job from `jobTemplate`.

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: backup
spec:
  schedule: "*/5 * * * *"          # every 5 minutes
  concurrencyPolicy: Forbid        # Allow (default) | Forbid | Replace
  startingDeadlineSeconds: 120     # skip a run if it can't start within 120s
  suspend: false                   # true = pause future runs
  successfulJobsHistoryLimit: 3    # keep 3 completed Jobs (default 3)
  failedJobsHistoryLimit: 1        # keep 1 failed Job (default 1)
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: backup
              image: busybox
              command: ["sh","-c","echo backing up; sleep 10"]
```

Cron schedule syntax

Five space-separated fields, in order: **minute hour day-of-month month day-of-week**.

Expression	Meaning
`* * * * *`	Every minute

`*/5 * * * *`	Every 5 minutes
`0 * * * *`	Top of every hour
`0 2 * * *`	Daily at 02:00
`0 0 * * 0`	Weekly, Sunday midnight
`0 9 1 * *`	09:00 on the 1st of each month

> 📌 **NOTE** CronJob times are interpreted in the `kube-controller-manager` 's time zone (UTC by default) unless you set `spec.timeZone` (1.27+ stable, e.g. `"America/New_York"`).

concurrencyPolicy & startingDeadlineSeconds

- **Allow** — overlapping runs are permitted (Jobs may pile up if one runs long).
- **Forbid** — skip the new run if the previous Job is still active.
- **Replace** — cancel the running Job and start a fresh one.

`startingDeadlineSeconds` is the grace window for a missed schedule (e.g. the controller was down). If the Job cannot start within it, that run is counted as missed and skipped. Miss more than 100 schedules in a row and the CronJob stops scheduling entirely — a good reason to set this field.

Imperative creation

```bash
# Job
k create job pi --image=perl:5.34 -- perl -Mbignum=bpi -wle 'print bpi(200)'
k create job pi --image=perl --dry-run=client -o yaml > job.yaml

# Job from an existing CronJob (manual trigger)
k create job manual-backup --from=cronjob/backup

# CronJob
k create cronjob backup --image=busybox --schedule="*/5 * * * *" -- sh -c "echo hi"
k create cronjob backup --image=busybox --schedule="0 2 * * *" \
  --dry-run=client -o yaml > cron.yaml
```

Inspecting & controlling

```bash
k get jobs
k get cronjob backup           # LAST SCHEDULE / ACTIVE / SUSPEND columns
k describe job pi              # Completions, failures, events
k logs job/pi                  # logs of a Job's Pod

# Pause / resume a CronJob without deleting it
k patch cronjob backup -p '{"spec":{"suspend":true}}'
k patch cronjob backup -p '{"spec":{"suspend":false}}'
```

Exam-day checklist

- Job Pod template **must** use `restartPolicy: Never` or `OnFailure` — `Always` is invalid.
- `completions` = how many must succeed; `parallelism` = how many at once; `backoffLimit` = retry cap before Failed.
- Use `k create job --from=cronjob/NAME` to trigger a CronJob run on demand.
- `suspend: true` pauses a CronJob; existing running Jobs are unaffected.
- Cron order is **min hour dom month dow**; times default to UTC unless `spec.timeZone` is set.
- Set `ttlSecondsAfterFinished` to auto-clean finished Jobs and avoid clutter.

Scheduling: nodeSelector & Node Affinity

When you create a Pod you rarely say *where* it should run — the **kube-scheduler** decides. But you often need to **influence** that decision: keep a Pod off control-plane nodes, pin it to GPU hardware, or spread replicas across zones. This chapter covers the tools that steer placement — `nodeName`, `nodeSelector`, node affinity, and pod (anti-)affinity — and, just as important for the exam, how to explain why a Pod is stuck `Pending`.

How the scheduler works

For each unscheduled Pod (one with empty `spec.nodeName`), the scheduler runs a two-phase pipeline:

1. **Filtering (predicates)** — eliminate nodes that cannot run the Pod: insufficient CPU/memory, unsatisfied `nodeSelector`/affinity, untolerated taints, port conflicts, unbound volume topology. The survivors are "feasible" nodes.
2. **Scoring (priorities)** — rank the feasible nodes (least-requested resources, image locality, spreading, affinity preferences). The highest-scoring node wins; ties break randomly.

The scheduler then **binds** the Pod by writing `nodeName`. If *no* node survives filtering, the Pod stays `Pending` and the scheduler records a `FailedScheduling` event.

nodeName — bypassing the scheduler

Setting `spec.nodeName` directly assigns the Pod to that node with **no** scheduling logic — no resource checks, no taint checks. The kubelet on that node just runs it. Useful for debugging, dangerous in production.

```yaml
apiVersion: v1
kind: Pod
metadata: {name: pinned}
spec:
  nodeName: worker-2      # skips the scheduler entirely
  containers:
    - name: c
      image: nginx
```

> ⚠️ **GOTCHA** If you set `nodeName` to a node without capacity, the Pod is placed anyway and may be evicted or stuck as the kubelet fails to admit it — the scheduler never protected it.

nodeSelector — the simplest targeting

A hard requirement: the Pod runs only on nodes carrying **all** the listed labels. Label a node first, then select it.

```bash
k label node worker-1 disktype=ssd
k get nodes --show-labels
k get nodes -l disktype=ssd
```

```yaml
spec:
  nodeSelector:
    disktype: ssd         # exact match, ALL keys must match
```

Node affinity — expressive targeting

Node affinity generalises `nodeSelector` with operators (`In` , `NotIn` , `Exists` , `DoesNotExist` , `Gt` , `Lt`) and two modes:

- **requiredDuringSchedulingIgnoredDuringExecution** — a hard rule; no matching node means the Pod stays Pending. "IgnoredDuringExecution" means a later label change does not evict an already-running Pod.
- **preferredDuringSchedulingIgnoredDuringExecution** — a soft rule with a `weight` (1–100); the scheduler prefers matching nodes but will place elsewhere if none match.

```yaml
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: disktype
                operator: In
                values: ["ssd","nvme"]
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 50
          preference:
            matchExpressions:
              - key: topology.kubernetes.io/zone
                operator: In
                values: ["us-east-1a"]
```

> 📌 **NOTE** Within `nodeSelectorTerms` , multiple **terms** are OR'd, but multiple `matchExpressions` inside one term are AND'd. Get this backwards and a Pod that should schedule stays Pending.

Pod affinity & anti-affinity

Where node affinity targets node labels, **pod affinity** targets other **Pods**: "schedule me near/away from Pods matching this label". This drives co-location (put the web Pod near its cache) and spreading (keep replicas apart for HA).

```yaml
spec:
  affinity:
    podAntiAffinity:                    # spread replicas across nodes
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchLabels: {app: web}
          topologyKey: kubernetes.io/hostname
    podAffinity:                        # co-locate with cache in same zone
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchLabels: {app: cache}
            topologyKey: topology.kubernetes.io/zone
```

topologyKey — the domain of the rule

`topologyKey` is a node label that defines the "region" over which the rule applies. The scheduler groups nodes by that label's value and enforces the rule per group:

topologyKey	Rule domain
`kubernetes.io/hostname`	Per node (one Pod per node)
`topology.kubernetes.io/zone`	Per availability zone
`topology.kubernetes.io/region`	Per region

> 🎯 **EXAM TIP** Anti-affinity with `topologyKey: kubernetes.io/hostname` is the standard "no two replicas on the same node" pattern. If you request more replicas than nodes with a **required** rule, the surplus Pods stay Pending — pods cannot double up.

Why is my Pod Pending?

Always start with the events.

```bash
k describe pod <name> | tail -n 20     # look for FailedScheduling
k get events --field-selector reason=FailedScheduling
k get nodes --show-labels               # do labels match the selector/affinity?
k describe node <node>                  # Taints + Allocatable resources
```

Message	Likely cause
didn't match Pod's node affinity/selector	No node has the required label
had untolerated taint	Nodes are tainted; Pod lacks toleration
Insufficient cpu/memory	No node has enough allocatable resources
had volume node affinity conflict	PV is zonal; feasible node is in another zone

Exam-day checklist

- Label a node with `k label node NAME key=val` before using `nodeSelector`; remove with `k label node NAME key-`.
- `required ...` = hard (Pending if unmet); `preferred ...` = soft (weighted, best-effort).
- Node affinity matches **node** labels; pod affinity matches **Pod** labels and always needs a `topologyKey`.
- Inside one selector term, `matchExpressions` are AND'd; across terms they are OR'd.
- `nodeName` bypasses the scheduler and all its checks — use only to force placement.
- Pending Pod? `k describe pod` first — the event names the exact filter that failed.

Taints & Tolerations

Node affinity and `nodeSelector` let a Pod **choose** nodes. Taints do the opposite from the node's side: a **taint** lets a node **repel** Pods that don't explicitly accept it. A **toleration** on a Pod is that acceptance. This push-pull pair is how Kubernetes keeps control-plane nodes clear of user workloads, reserves specialised hardware, and drains nodes gracefully. On the CKA you will taint a node, tolerate a taint, and explain why a Pod was evicted or refused.

The mental model: repel vs attract

Think of it as a bouncer. The **taint** is the node saying "you can't come in unless you have a pass". The **toleration** is the Pod's pass. Crucially, a toleration only *allows* a Pod onto a tainted node — it does **not** force it there. To attract a Pod to specific nodes you still need `nodeSelector`/affinity. Taints and affinity solve opposite halves of the placement problem and are frequently combined.

Taint anatomy and effects

A taint is `key=value:effect`. The **effect** determines how hard the node repels:

Effect	Behaviour
`NoSchedule`	New Pods without a matching toleration are refused. Existing Pods stay.
`PreferNoSchedule`	Soft version — scheduler tries to avoid the node but may still place there.
`NoExecute`	Refuses new Pods **and evicts** already-running Pods that don't tolerate it.

Adding, viewing, and removing taints

BASH

```
# Add a taint
k taint nodes worker-1 gpu=true:NoSchedule

# View a node's taints
k describe node worker-1 | grep -A2 Taints
k get node worker-1 -o jsonpath='{.spec.taints}'

# Remove a taint (note the trailing minus)
k taint nodes worker-1 gpu=true:NoSchedule-
k taint nodes worker-1 gpu-            # remove all taints with key 'gpu'
```

Tolerations on the Pod

A toleration must match the taint's `key`, `value`, and `effect`. The `operator` is either `Equal` (default — key and value must match) or `Exists` (match any value / just the key).

YAML

```
apiVersion: v1
kind: Pod
metadata: {name: gpu-pod}
spec:
  tolerations:
    - key: "gpu"
```

```
    operator: "Equal"       # Equal | Exists
    value: "true"
    effect: "NoSchedule"
containers:
  - name: c
    image: nginx
```

YAML

```
tolerations:
  - key: "gpu"
    operator: "Exists"      # tolerate ANY value for key 'gpu'
    effect: "NoSchedule"
  - operator: "Exists"      # empty key + Exists = tolerate EVERYTHING
```

⚠️ **GOTCHA** A toleration only makes the Pod *eligible* for the tainted node — it does not stop the scheduler from placing it on an untainted node instead. Pair with `nodeSelector` /node affinity if the Pod must land on the special node.

NoExecute and tolerationSeconds

`NoExecute` also evicts running Pods. A Pod can survive a while with `tolerationSeconds`: it stays for that many seconds after the taint appears, then is evicted. Omit the field to tolerate **forever**.

YAML

```
tolerations:
  - key: "node.kubernetes.io/not-ready"
    operator: "Exists"
    effect: "NoExecute"
    tolerationSeconds: 300     # evict 5 min after node goes NotReady
```

📌 **NOTE** Kubernetes auto-adds `NoExecute` tolerations for `node.kubernetes.io/not-ready` and `unreachable` with `tolerationSeconds: 300` to every Pod. That 300s is why a Pod lingers ~5 minutes before being rescheduled after a node fails.

The control-plane taint

kubeadm taints control-plane nodes so user workloads stay off them:

BASH

```
node-role.kubernetes.io/control-plane:NoSchedule
```

To run a Pod there (or a DaemonSet that must cover all nodes) add the matching toleration:

YAML

```
tolerations:
  - key: "node-role.kubernetes.io/control-plane"
    operator: "Exists"
    effect: "NoSchedule"
```

On a single-node cluster you commonly remove this taint so ordinary Pods can schedule:

BASH

```
k taint nodes --all node-role.kubernetes.io/control-plane-
```

Combining taints with affinity — dedicated nodes

To truly reserve nodes for one workload you need **both** mechanisms:

1. **Taint** the nodes so other Pods are repelled: `k taint nodes gpu-1 dedicated=gpu:NoSchedule`.
2. **Tolerate** the taint on the GPU Pods so they are allowed on.
3. **Node affinity / nodeSelector** on the GPU Pods so they are attracted *only* to those nodes (toleration alone would let them schedule elsewhere too).

> 🎯 **EXAM TIP** Taint = "keep others out". Affinity = "pull me in". You need the taint to exclude other workloads and the affinity/selector to confine your workload — one without the other leaves a gap.

How taints repel: interaction with the scheduler

During the scheduler's **filtering** phase, any node whose taints are not fully tolerated by the Pod is removed from the feasible set. `PreferNoSchedule` is not a filter — it applies a scoring penalty instead, so such nodes remain feasible but disfavoured. `NoExecute` is enforced continuously by the node controller, not just at scheduling time, which is why it can evict.

Exam-day checklist

- Taint syntax: `k taint nodes NODE key=value:EFFECT`; remove with a trailing `-`.
- Effects: `NoSchedule` (block new), `PreferNoSchedule` (soft), `NoExecute` (block new + evict existing).
- `operator: Exists` with an empty `key` tolerates every taint — handy for cluster-critical DaemonSets.
- `tolerationSeconds` only applies to `NoExecute`; omit it to tolerate indefinitely.
- A toleration permits but never forces placement — add affinity/selector for dedicated nodes.
- Pod Pending with "had untolerated taint"? Add the matching toleration or remove the taint.

Requests, Limits & Resource Management

Every container competes for the same finite CPU and memory on a node. Kubernetes uses two numbers per resource — a **request** and a **limit** — to decide *where* a Pod lands and *how much* it may consume once running. Get these wrong and you either waste capacity, get throttled, or watch the kernel kill your process. Resource management underpins scheduling, Quality-of-Service, eviction, and the Autoscaling and Namespace-governance topics on the CKA.

Requests vs limits — two jobs, one spec

A **request** is the amount the scheduler *reserves* for the container: it is the number used to find a node with enough allocatable capacity, and it never changes once the Pod is placed. A **limit** is the hard ceiling the kubelet/kernel enforces at runtime. Requests answer "will it fit?"; limits answer "how much is it allowed to grab?"

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web
spec:
  containers:
    - name: app
      image: nginx
      resources:
        requests:
          cpu: "250m"          # 250 millicores = 0.25 of a core
          memory: "128Mi"      # scheduler reserves this
        limits:
          cpu: "500m"          # throttled above this
          memory: "256Mi"      # OOMKilled above this
```

📌 **NOTE** CPU is measured in **millicores**: `1000m` = 1 vCPU/core. Memory uses binary suffixes — `Mi` (mebibyte, 2^20) and `Gi` — or decimal `M` / `G`. `128Mi` and `128M` are **not** equal; the exam expects the `Mi` / `Gi` forms.

How requests drive scheduling

The scheduler sums the `requests` of all Pods already on a node and only places a new Pod where `allocatable - sum(requests) ≥ Pod requests`. It never looks at actual live usage or at limits during this filter. So a node can be scheduled "full" on requests while sitting nearly idle, and a Pod with no request can be packed anywhere. If nothing fits, the Pod stays `Pending` with a `FailedScheduling` event citing *Insufficient cpu/memory*.

Limits: CPU is compressible, memory is not

This distinction is the single most tested resource concept. **CPU** is a *compressible* resource: exceed the limit and the kernel CFS quota simply **throttles** the container — it runs slower but keeps living. **Memory** is *incompressible*: you cannot "slow down" RAM usage. Exceed the memory limit and the kernel OOM-killer terminates the process. You will see the container restart with `Reason: OOMKilled` and exit code `137`.

⚠️ **GOTCHA** A container repeatedly hitting its memory limit shows up as `CrashLoopBackOff` with `Last State: Terminated, Reason: OOMKilled` in `k describe pod`. The fix is to raise the memory *limit* (or fix the leak) —

never the CPU limit.

Quality of Service (QoS) classes

The kubelet assigns each Pod a QoS class from how its requests and limits are set. QoS is not something you write — it is **derived**, and it decides eviction order when a node runs out of memory.

QoS class	Condition	Eviction priority
Guaranteed	Every container has requests = limits for *both* CPU and memory	Evicted last
Burstable	At least one container has a request/limit set, but not the Guaranteed criteria	Middle
BestEffort	No requests or limits set anywhere in the Pod	Evicted first

```bash
k get pod web -o jsonpath='{.status.qosClass}{"\n"}'   # Guaranteed | Burstable | BestEffort
```

🎯 **EXAM TIP** Under **node memory pressure** the kubelet evicts Pods in order: BestEffort first, then Burstable that exceed their requests, and Guaranteed last. Within a class, Pods using the most memory *over* their request go first. To make a Pod Guaranteed, set requests equal to limits for both resources.

LimitRange — per-namespace defaults and bounds

A **LimitRange** is a namespaced object that injects default requests/limits into containers that omit them, and enforces min/max per container. It is how an admin stops BestEffort Pods from ever appearing in a namespace.

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
  namespace: dev
spec:
  limits:
    - type: Container
      default:              # applied as the LIMIT if none given
        cpu: "500m"
        memory: "256Mi"
      defaultRequest:       # applied as the REQUEST if none given
        cpu: "250m"
        memory: "128Mi"
      max:                  # a container may not exceed these
        cpu: "1"
        memory: "512Mi"
      min:
        cpu: "100m"
        memory: "64Mi"
```

📌 **NOTE** A LimitRange only mutates Pods created **after** it exists — it does not retroactively patch running Pods. If a Pod's own values violate `min` / `max`, its creation is rejected with a validation error.

ResourceQuota — per-namespace aggregate caps

A **ResourceQuota** caps the *total* resources a namespace may consume across all Pods — aggregate CPU/memory requests and limits, plus object counts. If a namespace has a quota on a resource, every Pod in it **must** specify that resource or creation is refused (this is why quotas and LimitRanges are usually deployed together).

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-quota
  namespace: dev
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    pods: "20"
    count/services.loadbalancers: "2"
```

```bash
k describe quota team-quota -n dev     # shows Used vs Hard for each resource
k get resourcequota -n dev
```

⚠️ **GOTCHA** If a ResourceQuota sets `requests.cpu` / `limits.memory` etc., any Pod that omits those fields is rejected: *failed quota: must specify limits.memory*. Pair the quota with a LimitRange so defaults fill the gap.

Watching real usage: kubectl top

`kubectl top` reports *live* consumption (not requests/limits) and needs the **metrics-server** add-on installed — the same dependency the HPA relies on.

```bash
k top nodes                    # per-node CPU/memory usage
k top pods -A                  # per-Pod usage across namespaces
k top pod web --containers     # break down by container

# error 'metrics not available' == metrics-server missing/not ready
k -n kube-system get deploy metrics-server
```

Generating specs fast

```bash
# Scaffold a Pod then edit in resources:
k run web --image=nginx --dry-run=client -o yaml > web.yaml

# One-liner for a run with limits (requests default to the limit values):
k run web --image=nginx \
  --requests='cpu=250m,memory=128Mi' \
  --limits='cpu=500m,memory=256Mi' --dry-run=client -o yaml
```

Exam-day checklist

- Pod stuck `Pending` + *Insufficient cpu/memory* → the sum of **requests** on every node exceeds allocatable. Lower requests or add capacity.
- Container `OOMKilled` (exit 137) → memory limit too low. CPU pressure never kills, it only **throttles**.
- QoS is derived: requests == limits (both resources, every container) ⇒ **Guaranteed**; nothing set ⇒ **BestEffort** (evicted first).
- `k top` failing means **metrics-server** is absent — a common distractor.
- Quota rejects a Pod that omits a limited resource; a LimitRange supplies the missing default. Know that they are namespaced.

Autoscaling: HPA, VPA & Cluster Autoscaler

Kubernetes scales at three levels. The **HorizontalPodAutoscaler (HPA)** adds or removes Pod *replicas* based on observed load. The **VerticalPodAutoscaler (VPA)** right-sizes a Pod's requests/limits. The **Cluster Autoscaler** adds or removes *nodes* when Pods can't be scheduled. The HPA is the one you will be asked to create and reason about on the CKA; the other two you must recognise and describe.

HorizontalPodAutoscaler — scale out, scale in

An HPA targets a scalable workload (Deployment, ReplicaSet, StatefulSet) and drives its `replicas` up or down to keep an observed metric near a target. The controller loops every ~15s, reading metrics and applying the formula:

```bash
desiredReplicas = ceil( currentReplicas * ( currentMetric / targetMetric ) )
```

Example: 3 replicas averaging 90% CPU against a 50% target → `ceil(3 * 90/50) = ceil(5.4) = 6` replicas.

> ⚠️ **GOTCHA** The HPA depends entirely on the **metrics-server** add-on for CPU/memory utilisation. No metrics-server → HPA shows `<unknown>/50%` in the TARGETS column and never scales. This is the number-one HPA gotcha on the exam.

Utilisation is relative to the request

A CPU target of `50%` means 50% *of the Pod's CPU request* — not of a whole core. If a container requests `200m` and uses `100m`, that is 50% utilisation. Therefore **every container must define a CPU request** or the HPA cannot compute a percentage and will report `<unknown>`.

Imperative creation (fastest path)

```bash
# Autoscale a Deployment on CPU utilisation
k autoscale deployment web --cpu-percent=50 --min=2 --max=10

k get hpa                       # watch REPLICAS + TARGETS columns
k get hpa web -o yaml           # dump generated manifest
k describe hpa web              # events explain scaling decisions
```

HPA v2 YAML — CPU, memory & custom metrics

The `autoscaling/v2` API supports multiple metrics of type `Resource`, `Pods`, `Object`, or `External`. When several metrics are listed, the HPA computes a desired replica count for each and picks the **largest**.

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web
spec:
  scaleTargetRef:
    apiVersion: apps/v1
```

```
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization        # percent of request
          averageUtilization: 50
    - type: Resource
      resource:
        name: memory
        target:
          type: AverageValue       # absolute per-Pod value
          averageValue: 200Mi
```

📌 **NOTE** `Utilization` is a percentage of the request; `AverageValue` is an absolute quantity per Pod (e.g. requests-per-second from a custom metric). Custom/external metrics need an extra adapter (e.g. Prometheus Adapter) serving the `custom.metrics.k8s.io` API — beyond metrics-server.

Scaling behavior & stabilization

To stop replica counts oscillating ("flapping"), v2 exposes a `behavior` block with separate `scaleUp` / `scaleDown` policies and a **stabilization window**. During scale-down the HPA looks back over the window and uses the *highest* recommendation, so it shrinks slowly. The default scale-down window is 300s; scale-up is 0s (reacts immediately).

YAML

```
behavior:
  scaleDown:
    stabilizationWindowSeconds: 300   # wait before shrinking
    policies:
      - type: Percent
        value: 50                     # remove at most 50% of Pods
        periodSeconds: 60
  scaleUp:
    stabilizationWindowSeconds: 0
    policies:
      - type: Pods
        value: 4                       # add at most 4 Pods
        periodSeconds: 60
```

🎯 **EXAM TIP** If a Deployment is scaled by an HPA, do **not** set `replicas` in the Deployment manifest or run `kubectl scale` — the HPA owns that field and will fight you. The two controllers competing is a classic cause of "replicas keep changing."

VerticalPodAutoscaler (VPA) — a note

The VPA adjusts a Pod's CPU/memory **requests and limits** instead of the replica count. It runs three components (Recommender, Updater, Admission Controller) and is **not built into core Kubernetes** — it is a separate add-on.

In `Auto` mode it must *evict and recreate* Pods to apply new sizes, because a running Pod's resources have historically been immutable. Crucially, **the VPA and HPA must not manage the same resource metric on the same workload** (e.g. both on CPU) — they conflict. A common pattern is HPA on a custom metric while VPA tunes memory.

Cluster Autoscaler — scaling nodes

When Pods stay `Pending` because no node has room, the **Cluster Autoscaler** adds nodes by growing the cloud provider's node group (AWS ASG, GKE node pool, etc.). Conversely it removes a node when its Pods can be rescheduled elsewhere and it has been under-utilised for ~10 minutes.

Autoscaler	Scales	Trigger signal	Built-in?
HPA	Pod replica count	Metric vs target (CPU/mem/custom)	Yes
VPA	Pod requests/limits	Historical usage recommendation	No (add-on)
Cluster Autoscaler	Node count	Unschedulable (Pending) Pods	No (add-on)

> 📌 **NOTE** The Cluster Autoscaler reacts to Pods it *cannot* schedule, so accurate **requests** are essential — a Pod with no request may pack onto a full node, and one with an oversized request may trigger a needless node. It respects PodDisruptionBudgets when draining a node for scale-down.

Troubleshooting an HPA

```bash
k describe hpa web
# 'unable to get metrics for resource cpu' → metrics-server down, or
# the target Pods have no cpu request set.

k top pods                      # confirm metrics-server actually serves data
k get hpa web                   # TARGETS shows <unknown>/50% when broken
```

Exam-day checklist

- `k autoscale deployment NAME --cpu-percent=50 --min=2 --max=10` — memorise this one-liner.
- TARGETS shows `<unknown>` ⇒ missing **metrics-server** or missing CPU **request** on the Pods.
- Utilisation targets are a **percentage of the request**, not of a core.
- With multiple metrics the HPA takes the **maximum** desired replica count.
- Do not hard-code `replicas` on an HPA-managed Deployment.
- HPA = replicas, VPA = Pod size, Cluster Autoscaler = node count. VPA and Cluster Autoscaler are add-ons, not core.

Health Probes: Liveness, Readiness & Startup

The kubelet cannot know whether the process inside a container is actually healthy or merely running. **Probes** close that gap: they let you tell Kubernetes how to test a container and what to do when the test fails. The three probe types answer three different questions — *should I restart it?*, *should I send it traffic?*, and *has it finished booting?* — and confusing them is a favourite exam trap.

The three probe types

Probe	Question	Action on failure
livenessProbe	Is the container still healthy?	Kubelet **restarts** the container
readinessProbe	Can it serve requests right now?	**Removed from Service endpoints** (not restarted)
startupProbe	Has it finished starting up?	Disables the other probes until it succeeds; on failure, restarts

⚠️ **GOTCHA** The critical distinction: a failing **liveness** probe **kills and restarts** the container; a failing **readiness** probe only pulls the Pod out of Service load-balancing — the container keeps running and can recover. Using liveness where you meant readiness causes needless restart loops.

Readiness and why it matters for Services & rollouts

A Pod's IP is added to a Service's `Endpoints` (EndpointSlice) only while its readiness probe passes. During a rolling update, the Deployment controller waits for new Pods to become **Ready** before scaling down old ones — so readiness gates the entire rollout. Without a readiness probe, Kubernetes treats a container as Ready the instant it starts, and traffic hits a process that may still be loading. This is the mechanism behind zero-downtime deploys.

🎯 **EXAM TIP** If a rolling update stalls with new Pods stuck `0/1 Running`, the readiness probe is failing. `k describe pod` shows *Readiness probe failed* events — check the path, port, and `initialDelaySeconds`.

Probe handlers

Every probe uses one of four handler mechanisms. Exactly one per probe.

Handler	Succeeds when	Typical use
`httpGet`	HTTP status 200–399	Web servers / REST APIs
`tcpSocket`	TCP connection opens	Databases, non-HTTP services
`exec`	Command exits with code 0	Custom checks, CLIs, files
`grpc`	gRPC health check responds SERVING	gRPC services (1.24+ stable)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web
```

```yaml
spec:
  containers:
    - name: app
      image: nginx
      ports:
        - containerPort: 80
      livenessProbe:
        httpGet:
          path: /healthz
          port: 80              # name or number; add httpHeaders if needed
        initialDelaySeconds: 10
        periodSeconds: 10
        timeoutSeconds: 1
        failureThreshold: 3
      readinessProbe:
        tcpSocket:
          port: 80
        initialDelaySeconds: 5
        periodSeconds: 5
        successThreshold: 1
```

YAML

```yaml
      # exec and grpc handler examples
      livenessProbe:
        exec:
          command: ["cat", "/tmp/healthy"]   # exit 0 = pass
      readinessProbe:
        grpc:
          port: 9000
          service: readiness    # optional gRPC service name
```

Timing fields — the knobs

Field	Default	Meaning
`initialDelaySeconds`	0	Wait after container start before first probe
`periodSeconds`	10	How often to probe
`timeoutSeconds`	1	Probe timeout before counting as a failure
`failureThreshold`	3	Consecutive failures before acting
`successThreshold`	1	Consecutive successes to be considered passing (must be 1 for liveness/startup)

> 📌 **NOTE** Worst-case time to detect a dead container ≈ `initialDelaySeconds + (failureThreshold × periodSeconds)` . With the defaults above (10 + 3×10) the kubelet restarts a hung container roughly 40s after it stops responding.

The startup probe — for slow starters

Legacy or JVM apps may take a minute to boot. If you set a liveness probe aggressive enough for steady state, it will kill the container mid-boot before it ever comes up. The **startupProbe** solves this: while it is running, liveness

and readiness probes are *disabled*. Only once the startup probe succeeds do the others begin. This lets you grant a long boot budget without weakening steady-state liveness.

```yaml
    startupProbe:
      httpGet:
        path: /healthz
        port: 80
      failureThreshold: 30    # 30 * 10s = up to 300s to start
      periodSeconds: 10
    livenessProbe:
      httpGet:
        path: /healthz
        port: 80
      periodSeconds: 10       # tight steady-state check, safe now
```

Inspecting & debugging probes

```bash
# Restart count climbing? liveness is failing.
k get pod web                          # RESTARTS column
k describe pod web                     # 'Liveness/Readiness probe failed' events

# Reproduce the check from inside the container
k exec web -- curl -sf localhost:80/healthz
k exec web -- cat /tmp/healthy

# Endpoints reflect readiness
k get endpoints my-svc                 # missing IP = Pod not Ready
```

Common gotchas

- **Liveness restart loop**: the app is slow to start, not unhealthy. Add a `startupProbe` or raise `initialDelaySeconds`.
- **Probe port must be exposed by the container**, and for `httpGet` the path must return 2xx/3xx. A 404 counts as a failure.
- `successThreshold` must be **1** for liveness and startup probes; only readiness may set it higher.
- Probes run *inside* the Pod network namespace — `httpGet host` defaults to the Pod IP, so use `localhost`/pod-local ports, not the Service name.
- An `exec` probe forks a process every period — heavy commands add load; keep them cheap.

Exam-day checklist

- Restart storm ⇒ suspect **liveness**; Pod stays `0/1` but running ⇒ suspect **readiness**.
- Slow-booting app killed at startup ⇒ add a **startupProbe**.
- Readiness gates Service endpoints and rolling updates — no readiness means instant (possibly premature) traffic.
- Know the four handlers (`httpGet` / `tcpSocket` / `exec` / `grpc`) and the five timing fields with their defaults.

ConfigMaps & Secrets

The twelve-factor rule is to keep configuration out of the image. Kubernetes provides two objects for this: **ConfigMaps** for non-sensitive key/value data and **Secrets** for sensitive data. They share almost identical mechanics — the same consumption patterns, the same volume behaviour — so learning one teaches the other. Both are namespaced and both are surprisingly easy to misuse, especially around the myth that Secrets are "encrypted."

Creating a ConfigMap

You rarely hand-write a ConfigMap in the exam — the imperative `create` subcommands are faster, and adding `--dry-run=client -o yaml` gives you a manifest to tweak.

BASH

```bash
# From individual literals
k create configmap app-config \
  --from-literal=LOG_LEVEL=debug \
  --from-literal=MODE=prod

# From a file (key = filename, value = file contents)
k create configmap nginx-conf --from-file=nginx.conf

# Choose the key explicitly
k create configmap nginx-conf --from-file=conf=nginx.conf

# From a whole directory (one key per file)
k create configmap certs --from-file=./config-dir/

# From an env file (each KEY=VALUE line becomes a key)
k create configmap env-config --from-env-file=app.env

# Generate YAML to edit
k create configmap app-config --from-literal=LOG_LEVEL=debug \
  --dry-run=client -o yaml
```

YAML

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: debug
  MODE: prod
  app.properties: |          # multi-line value (a config file)
    server.port=8080
    cache.enabled=true
```

📌 **NOTE** `--from-file` makes the **filename the key** and the file contents the value. `--from-env-file` parses each `KEY=VALUE` *line* into a separate key. Mixing these up is the most common ConfigMap mistake.

Consuming config: env vars vs volume mounts

There are two ways to get data into a container, and the choice matters for live updates.

As environment variables

```yaml
spec:
  containers:
    - name: app
      image: nginx
      env:
        - name: LOG_LEVEL                # pick one key
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: LOG_LEVEL
      envFrom:
        - configMapRef:
            name: app-config             # import ALL keys as env vars
        - secretRef:
            name: db-secret
```

As a volume mount

```yaml
spec:
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - name: config
          mountPath: /etc/config         # each key becomes a file here
  volumes:
    - name: config
      configMap:
        name: app-config
        items:                           # optional: project chosen keys
          - key: app.properties
            path: app.properties
```

> 🎯 **EXAM TIP** A ConfigMap/Secret mounted as a **volume** updates in the container automatically (within ~1 min) when the object changes — but data injected as **environment variables** is fixed at Pod start and only refreshes on a Pod restart. Use volume mounts if you need live reloads.

subPath — mount a single file without hiding the directory

Mounting a volume at `/etc/nginx` replaces the whole directory. To drop in just one file while keeping the rest of the directory, use `subPath`.

```yaml
      volumeMounts:
        - name: config
          mountPath: /etc/nginx/nginx.conf   # single file
          subPath: nginx.conf
```

> ⚠️ **GOTCHA** A `subPath` mount does **not** receive automatic updates when the ConfigMap changes — it is copied once at mount time. This is a deliberate trade-off and a known exam gotcha: use a plain directory mount if you need live updates.

Secrets — same shape, sensitive data

Secrets look like ConfigMaps but values are **base64-encoded** in the API and get extra handling (kept out of some logs, mountable as tmpfs). There are several typed Secrets:

Type	Purpose
`Opaque`	Default; arbitrary user key/value data
`kubernetes.io/dockerconfigjson`	Private registry pull credentials
`kubernetes.io/tls`	TLS cert + key (`tls.crt` / `tls.key`)
`kubernetes.io/service-account-token`	ServiceAccount token

```bash
# Generic (Opaque) secret
k create secret generic db-secret \
  --from-literal=username=admin \
  --from-literal=password='S3cr3t!'

# TLS secret from cert/key files
k create secret tls web-tls --cert=tls.crt --key=tls.key

# Registry credentials
k create secret docker-registry regcred \
  --docker-server=registry.example.com \
  --docker-username=me --docker-password=pw
```

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  username: YWRtaW4=          # base64("admin")
stringData:
  password: "S3cr3t!"         # plaintext here; API encodes it for you
```

> 📌 **NOTE** Use `stringData` to write plaintext in a manifest — Kubernetes base64-encodes it into `data` on creation. Decode with `k get secret db-secret -o jsonpath='{.data.username}' | base64 -d` .

Base64 is NOT encryption

The single most important Secret fact: base64 is an **encoding**, trivially reversible by anyone with read access to the object or to etcd. A Secret is no more secret than a ConfigMap unless you take further steps.

> ⚠️ **GOTCHA** Anyone who can `get` a Secret, or read the etcd datastore, can decode it instantly with `base64 -d`. Base64 provides **zero** confidentiality. Protect Secrets with RBAC and encryption at rest.

Encryption at rest

By default Secrets are stored **unencrypted** in etcd. To encrypt them, the API server is started with `--encryption-provider-config` pointing to an `EncryptionConfiguration` that defines providers (e.g. `aescbc`, `aesgcm`, or an external KMS). Order matters: the first provider encrypts new writes; `identity` means plaintext.

```yaml
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
  - resources: ["secrets"]
    providers:
      - aescbc:
          keys:
            - name: key1
              secret: <base64-encoded-32-byte-key>
      - identity: {}          # fallback = no encryption
```

Immutable ConfigMaps & Secrets

Setting `immutable: true` prevents any further updates to the object's data. This protects against accidental changes and lets the kubelet stop watching the object, reducing API-server load at large scale. To change an immutable object you must delete and recreate it (and restart consuming Pods).

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
immutable: true
data:
  LOG_LEVEL: debug
```

Exam-day checklist

- `--from-literal` (inline), `--from-file` (filename=key), `--from-env-file` (one key per `KEY=VALUE` line) — know the difference cold.
- `env` / `envFrom` inject at start (need restart to change); **volume mounts auto-update** — except `subPath`, which does not.
- Base64 ≠ encryption. Real protection = RBAC + **encryption at rest** via `EncryptionConfiguration`.
- Registry pull creds use a `docker-registry` secret referenced by `imagePullSecrets`; TLS uses a `tls` secret.
- `immutable: true` blocks edits — delete/recreate to change.

Services: ClusterIP, NodePort & LoadBalancer

Pods are ephemeral and their IPs change on every restart or reschedule, so you can never hard-code a Pod IP. A **Service** is a stable virtual endpoint — a durable name and IP — that load-balances traffic across a dynamic set of backing Pods selected by label. Services are the glue of in-cluster networking and appear all over the CKA: exposing Deployments, wiring Ingress backends, and debugging why traffic never reaches a Pod.

The Service abstraction and how it finds Pods

A Service does not know about Pods directly. Its `spec.selector` is a label query; the controller watches for Pods matching that query and records their IP:port pairs as **Endpoints** (legacy) and **EndpointSlices** (the scalable modern form, default since 1.19). kube-proxy programs the node's dataplane from those slices. Break the label match and the Service still exists but has zero endpoints — the classic "Service returns connection refused" bug.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector:
    app: web                # matches Pods labelled app=web
  ports:
    - name: http
      port: 80              # the Service's own port (ClusterIP:80)
      targetPort: 8080      # the container port traffic is forwarded to
  # type defaults to ClusterIP
```

> 📌 **NOTE** A Service with **no selector** does not auto-populate endpoints. You then create a matching `Endpoints` / `EndpointSlice` by hand — the pattern for pointing a Service at an external database or a fixed IP.

port vs targetPort vs nodePort

These three are constantly confused and constantly tested. Keep them straight:

Field	Lives on	Meaning
`port`	The Service (ClusterIP)	Port clients hit on the virtual IP
`targetPort`	The Pod/container	Where traffic is delivered; defaults to `port` if omitted. May be a named port.
`nodePort`	Every node	External port opened on all nodes (NodePort/LoadBalancer only)

`targetPort` can name a port defined in the Pod spec (`containerPort: {name: http}`), which decouples the Service from the actual number — handy when different Pods listen on different ports.

Service types

ClusterIP (default)

An internal-only virtual IP reachable from inside the cluster. This is the base type; every other type is a superset built on top of it.

NodePort

Allocates a port on **every** node in the range `30000-32767` (configurable via `--service-node-port-range` on the API server). Traffic to `<anyNodeIP>:nodePort` is forwarded to the Service, then to a Pod. It still gets a ClusterIP too.

```yaml
apiVersion: v1
kind: Service
metadata: {name: web-np}
spec:
  type: NodePort
  selector: {app: web}
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080      # optional; omit and one is auto-assigned in 30000-32767
```

LoadBalancer

Provisions an external cloud load balancer (via the cloud controller manager) that fronts the NodePort. On bare metal it stays `Pending` unless something like MetalLB provides the external IP.

ExternalName

No proxying, no selector, no endpoints. It maps the Service name to an external DNS name by returning a CNAME record from CoreDNS.

```yaml
apiVersion: v1
kind: Service
metadata: {name: db}
spec:
  type: ExternalName
  externalName: db.prod.example.com   # CoreDNS returns a CNAME to this
```

Headless (clusterIP: None)

Set `clusterIP: None` and Kubernetes assigns no virtual IP and does no load-balancing. Instead, a DNS lookup of the Service returns the A records of all matching Pods directly. This is how StatefulSets give each Pod a stable, individually addressable identity.

```yaml
apiVersion: v1
kind: Service
metadata: {name: web-headless}
spec:
  clusterIP: None          # headless
  selector: {app: web}
  ports:
    - port: 80
```

kube-proxy modes: iptables vs ipvs

kube-proxy runs on every node and implements the Service virtual IP. It does not sit in the data path — it programs kernel rules and gets out of the way.

Mode	Mechanism	Notes
`iptables`	Chains of iptables rules; random pick per connection	Default; O(n) rule evaluation, fine for most clusters
`ipvs`	Kernel IPVS load balancer (hash tables)	Scales to thousands of Services; supports rr, lc, sh scheduling

> 🎯 **EXAM TIP** Both modes only distribute per-connection, not per-request. There is no weighting or app-layer awareness — that is what Ingress/Gateway or a service mesh add.

sessionAffinity

By default connections are spread across Pods. Set `spec.sessionAffinity: ClientIP` to pin a client IP to the same backend Pod (sticky sessions), with a configurable timeout.

```yaml
spec:
  sessionAffinity: ClientIP
  sessionAffinityConfig:
    clientIP:
      timeoutSeconds: 10800   # default 3h
```

Essential commands

```bash
# Fast create + generate YAML to edit
k expose deployment web --port=80 --target-port=8080
k expose deploy web --type=NodePort --port=80 --dry-run=client -o yaml > svc.yaml

# Create directly (no existing object needed)
k create service clusterip web --tcp=80:8080 --dry-run=client -o yaml

# Inspect and verify endpoints (the #1 debug step)
k get svc,endpointslices -o wide
k get endpoints web            # empty list == selector matches no Pods
k describe svc web             # shows Endpoints line + Session Affinity

# Test from inside the cluster
k run tmp --rm -it --image=busybox --restart=Never -- wget -qO- web:80
```

> ⚠️ **GOTCHA** If a Service has no endpoints, check three things in order: (1) does the `selector` match the Pod labels? (2) are the Pods `Ready` ? Only ready Pods become endpoints. (3) does `targetPort` match the port the container actually listens on?

Exam-day checklist

- NodePort range is **30000-32767** — memorise it; questions test the boundary.
- `k expose` is the fastest way to make a Service; add `--dry-run=client -o yaml` when you need to tweak `type` or `nodePort`.
- Headless = `clusterIP: None`; returns per-Pod A records, no load-balancing.
- Empty `Endpoints` is almost always a selector/label or readiness mismatch.
- LoadBalancer stuck `Pending` on bare metal is expected without MetalLB.

Ingress & the Gateway API

A Service of type LoadBalancer gives you one external IP per Service — expensive and crude. **Ingress** is an HTTP/HTTPS layer-7 router that lets one entry point fan out to many Services by host and path, with TLS termination in one place. The CKA now also covers the newer, more expressive **Gateway API**, which is the successor design. Expect questions on both the resource YAML and on why nothing works without a controller.

Ingress needs a controller

The `Ingress` object is just data — a set of routing rules stored in the API. Nothing happens until an **Ingress Controller** (NGINX, Traefik, HAProxy, a cloud LB controller, etc.) is running to watch those objects and configure a real proxy. A cluster with no controller will accept your Ingress YAML and quietly route nothing.

> ⚠️ **GOTCHA** Creating an Ingress on a cluster with no Ingress Controller installed is a silent no-op. There is no error — traffic simply never flows. Confirm a controller Pod is running before you debug rules.

The Ingress resource

YAML

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /   # controller-specific
spec:
  ingressClassName: nginx        # which controller handles this
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-svc
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web-svc
                port:
                  number: 80
```

pathType — the exam trap

pathType	Match behaviour
`Prefix`	Matches by URL path prefix split on `/`. `/api` matches `/api/x`.

`Exact`	Matches the URL path exactly, case-sensitive. `/api` does not match `/api/`.
`ImplementationSpecific`	Left to the controller — avoid on the exam unless told to use it.

`pathType` is **required** in `networking.k8s.io/v1`. A rule with no `host` matches all inbound hosts (a catch-all / default backend style rule).

TLS termination via a Secret

The controller decrypts HTTPS at the edge using a `kubernetes.io/tls` Secret holding `tls.crt` and `tls.key`. The `hosts` list must match the certificate's SAN and the rule host.

YAML

```yaml
spec:
  tls:
    - hosts:
        - shop.example.com
      secretName: shop-tls        # a kubernetes.io/tls Secret
```

BASH

```bash
# Create the TLS secret from cert/key files
k create secret tls shop-tls --cert=tls.crt --key=tls.key \
  --dry-run=client -o yaml > shop-tls.yaml
```

IngressClass

Multiple controllers can coexist. `IngressClass` names a controller; `spec.ingressClassName` on an Ingress selects it. One IngressClass can be marked default so Ingresses that omit the field use it.

YAML

```yaml
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
  name: nginx
  annotations:
    ingressclass.kubernetes.io/is-default-class: "true"
spec:
  controller: k8s.io/ingress-nginx
```

The Gateway API

Ingress conflates too many concerns into one object and pushes everything else into annotations. The **Gateway API** (group `gateway.networking.k8s.io`) splits responsibilities across role-oriented resources and is now in the CKA curriculum.

Resource	Owned by	Role
`GatewayClass`	Infra provider	Cluster-scoped template naming the controller implementation
`Gateway`	Cluster operator	An actual listener (ports, protocol, TLS) — the entry point
`HTTPRoute`	App developer	Host/path routing rules attached to a Gateway

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: prod-gw
spec:
  gatewayClassName: nginx
  listeners:
    - name: http
      protocol: HTTP
      port: 80
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: shop-route
spec:
  parentRefs:
    - name: prod-gw            # attach to the Gateway above
  hostnames:
    - "shop.example.com"
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /api
      backendRefs:
        - name: api-svc
          port: 80
```

🎯 **EXAM TIP** Mapping Ingress → Gateway API: an Ingress rule's `host` becomes `HTTPRoute.hostnames`; `path` / `pathType` become a `matches.path` with `type: PathPrefix` or `Exact`; `backend.service` becomes `backendRefs`; and TLS/ports move up to the `Gateway.listeners`.

Essential commands

```bash
# Fast Ingress scaffold
k create ingress shop --class=nginx \
  --rule="shop.example.com/api*=api-svc:80" \
  --rule="shop.example.com/*=web-svc:80" \
  --dry-run=client -o yaml > ing.yaml

k get ingress                # note the ADDRESS column fills once wired
k describe ingress shop      # shows rules, TLS, and controller events
k get ingressclass

# Gateway API (CRDs must be installed)
k get gatewayclass
k get gateway,httproute -A
```

Exam-day checklist

- No controller = no routing, with no error. Verify the controller Pod first.
- `pathType` is mandatory in v1; know `Prefix` vs `Exact`.
- TLS uses a `kubernetes.io/tls` Secret; the `hosts` must match the rule.
- `ingressClassName` (spec field) is preferred over the old `kubernetes.io/ingress.class` annotation.
- Gateway API: three objects — **GatewayClass** → **Gateway** → **HTTPRoute**. HTTPRoute attaches to a Gateway via `parentRefs`.

NetworkPolicies

By default Kubernetes networking is wide open: every Pod can talk to every other Pod in the cluster, across all namespaces. A **NetworkPolicy** is a namespaced firewall rule, expressed in Pod labels rather than IPs, that restricts which connections are allowed. This is one of the most exam-heavy Security/Networking topics — you will almost certainly write a default-deny plus an allow rule under time pressure.

Default allow, and how a policy flips it

The mental model that trips people up: policies are **additive and whitelist-only**. There is no "deny" rule. The logic is:

- If **no** policy selects a Pod, all traffic is allowed (the default).
- The moment **any** policy selects a Pod for a direction (ingress or egress), that Pod becomes **default-deny for that direction** — only what the policy explicitly allows gets through.
- Ingress and egress are independent. A policy with only `ingress` rules does not restrict egress at all, and vice versa.

> 📌 **NOTE** Selecting a Pod with an empty ingress rule set denies all ingress; selecting it with rules allows only those. You never "block" — you switch a Pod into deny-by-default and then poke holes.

A CNI must enforce them

The API server happily stores NetworkPolicy objects, but enforcement is the job of the CNI plugin. **Calico** and **Cilium** enforce them; the plain `kubenet` and some minimal setups do not. On such clusters your policy is accepted and does nothing.

> ⚠️ **GOTCHA** Like Ingress without a controller, a NetworkPolicy on a CNI that does not support it is a silent no-op. If a "block" question does not seem to take effect, confirm the cluster runs a policy-capable CNI (Calico/Cilium).

Default-deny all ingress

An empty `podSelector: {}` selects every Pod in the namespace; with no ingress rules, nothing is allowed in.

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: prod
spec:
  podSelector: {}            # all Pods in prod
  policyTypes:
    - Ingress                # deny all ingress; egress untouched
```

To lock down both directions, add `Egress` to `policyTypes` with no egress rules.

The building blocks: podSelector, namespaceSelector, ipBlock

Peer selector	Selects
`podSelector`	Pods (by label) in the **same** namespace as the policy
`namespaceSelector`	All Pods in namespaces matching the label
`ipBlock`	A CIDR range (with optional `except`) — for external/off-cluster IPs

Allow from a specific namespace

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-monitoring
  namespace: prod
spec:
  podSelector:
    matchLabels:
      app: web               # protect the web Pods
  policyTypes: [Ingress]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: monitoring
      ports:
        - protocol: TCP
          port: 8080
```

🎯 **EXAM TIP** Every namespace automatically carries the immutable label `kubernetes.io/metadata.name=<name>`. Use it to target a namespace by name without adding your own labels.

AND within a from-block vs OR across blocks

This distinction is the single most tested subtlety. Inside **one** `from` element, listing both a `namespaceSelector` and a `podSelector` means **AND** — the peer must be in that namespace AND match that Pod label. Listing them as **separate** list items under `from` means **OR**.

```yaml
  # AND: Pods labelled role=api that live in namespace team=blue
  ingress:
    - from:
        - namespaceSelector:
            matchLabels: {team: blue}
          podSelector:
            matchLabels: {role: api}

  # OR: (any Pod in team=blue namespaces) OR (role=api Pods in THIS namespace)
  ingress:
    - from:
        - namespaceSelector:
            matchLabels: {team: blue}
```

```
        - podSelector:
            matchLabels: {role: api}
```

The whole difference is one level of indentation and one `-`. Read the YAML carefully in the exam: two entries under `from` = OR; two selectors under one entry = AND.

Egress example with DNS

When you default-deny egress, remember Pods still need DNS. You must explicitly allow egress to CoreDNS on UDP/TCP 53, or name resolution breaks.

YAML

```yaml
spec:
  podSelector: {matchLabels: {app: web}}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
            matchLabels: {kubernetes.io/metadata.name: kube-system}
      ports:
        - {protocol: UDP, port: 53}
        - {protocol: TCP, port: 53}
    - to:
        - podSelector: {matchLabels: {app: db}}
      ports:
        - {protocol: TCP, port: 5432}
```

Essential commands

BASH

```bash
k get netpol -A
k describe netpol default-deny-ingress -n prod   # shows resolved selectors

# Verify connectivity before/after applying a policy
k run test --image=busybox -n prod --rm -it --restart=Never -- \
  wget -qO- --timeout=2 web:8080

# There is no kubectl create netpol generator — write YAML by hand
k apply -f netpol.yaml
```

Exam-day checklist

- No policy selects a Pod = all traffic allowed. First matching policy flips it to deny-by-default **for that direction only**.
- `podSelector: {}` = all Pods in the namespace; empty rules = deny all.
- Two items under `from` / `to` = **OR**; two selectors in one item = **AND**.
- `podSelector` is same-namespace only; cross-namespace needs a `namespaceSelector`.
- Default-deny egress breaks DNS — allow UDP/TCP 53 to kube-system.
- Policy has no effect unless the CNI (Calico/Cilium) enforces it.

CoreDNS & Service Discovery

Pods find each other by name, not IP, and the machinery behind that is cluster DNS. Since Kubernetes 1.13 the default DNS server is **CoreDNS**, running as a Deployment in `kube-system` and fronted by a Service called `kube-dns`. Understanding the FQDN pattern and how to debug resolution from inside a Pod is a reliable source of CKA marks — many "the app can't reach the service" problems are actually DNS problems.

How CoreDNS runs

CoreDNS is a Deployment (usually 2 replicas) plus a Service. The `kubelet` injects that Service's ClusterIP into every Pod's `/etc/resolv.conf` as the nameserver. Its configuration lives in a **ConfigMap** named `coredns` whose key is the **Corefile**.

```bash
k get deploy,svc,pods -n kube-system -l k8s-app=kube-dns
k get configmap coredns -n kube-system -o yaml
```

```yaml
# The Corefile (inside the coredns ConfigMap)
.:53 {
    errors
    health
    kubernetes cluster.local in-addr.arpa ip6.arpa {   # the k8s plugin
        pods insecure
        fallthrough in-addr.arpa ip6.arpa
    }
    forward . /etc/resolv.conf     # non-cluster names go upstream
    cache 30
    loop
    reload                         # picks up ConfigMap edits automatically
}
```

> 🎯 **EXAM TIP** The `reload` plugin means editing the ConfigMap takes effect within ~2 minutes without restarting Pods. If you must force it, `k rollout restart deploy coredns -n kube-system`.

The FQDN pattern

Every Service gets an A/AAAA record following a fixed hierarchy:

```bash
<service>.<namespace>.svc.<cluster-domain>
# e.g.
web.default.svc.cluster.local
```

From where	What you can type
Same namespace	`web`
Different namespace	`web.prod` or `web.prod.svc`
Fully qualified (always works)	`web.prod.svc.cluster.local`

Short names resolve because `/etc/resolv.conf` lists `search` domains and an `ndots:5` option, so the resolver tries appending the search suffixes before treating a name as absolute.

```bash
# Inside a Pod
cat /etc/resolv.conf
# nameserver 10.96.0.10
# search default.svc.cluster.local svc.cluster.local cluster.local
# options ndots:5
```

Pod DNS records

Pods also get A records under `pod.cluster.local`, with dots in the IP replaced by dashes: `10-244-1-5.default.pod.cluster.local`. These are rarely used directly except by headless-service StatefulSet Pods.

Headless Service DNS — per-Pod A records

A normal Service resolves to one ClusterIP. A **headless** Service (`clusterIP: None`) instead returns the individual A records of every ready backing Pod. Combined with a StatefulSet, each Pod gets a stable name:

```bash
<pod>.<headless-service>.<namespace>.svc.cluster.local
# e.g. web-0.web-headless.default.svc.cluster.local
```

This is what lets database clusters address peers by stable identity (`db-0`, `db-1`) rather than a load-balanced VIP.

dnsPolicy

Each Pod's `spec.dnsPolicy` controls which resolver it uses:

dnsPolicy	Behaviour
`ClusterFirst`	Default. Cluster DNS first; non-cluster names forwarded upstream.
`Default`	Inherit the node's `/etc/resolv.conf` — **not** cluster DNS.
`ClusterFirstWithHostNet`	Use cluster DNS even when `hostNetwork: true`.
`None`	Ignore defaults; you supply everything via `dnsConfig`.

> 📌 **NOTE** Trap: a Pod with `hostNetwork: true` and the default `ClusterFirst` policy silently falls back to the node's resolver. Set `dnsPolicy: ClusterFirstWithHostNet` so it still uses CoreDNS.

```yaml
spec:
  dnsPolicy: "None"
  dnsConfig:
    nameservers: ["1.1.1.1"]
    searches: ["example.com"]
    options:
      - {name: ndots, value: "2"}
```

Debugging DNS from a Pod

```bash
# Spin up a throwaway Pod with DNS tools
k run dns --image=busybox:1.28 --rm -it --restart=Never -- sh

# Inside:
nslookup web                    # short name (same ns)
nslookup web.prod.svc.cluster.local
nslookup kubernetes.default     # the API server Service always exists

# Is CoreDNS healthy?
k get pods -n kube-system -l k8s-app=kube-dns
k logs -n kube-system -l k8s-app=kube-dns
k get svc kube-dns -n kube-system    # ClusterIP must match Pods' resolv.conf
```

⚠️ **GOTCHA** Use `busybox:1.28` for `nslookup` — newer BusyBox images have a broken nslookup that misreports cluster DNS. This exact gotcha shows up in labs and the exam.

Exam-day checklist

- FQDN is `service.namespace.svc.cluster.local`; short names work via `search` domains and `ndots:5`.
- CoreDNS = Deployment in `kube-system` + `coredns` ConfigMap holding the Corefile; `kube-dns` is the Service name.
- Cross-namespace access needs at least `service.namespace`, not the bare name.
- Headless Service returns per-Pod A records; `pod.headless-svc.ns.svc.cluster.local` for stable StatefulSet identities.
- `hostNetwork` Pods need `ClusterFirstWithHostNet` to keep cluster DNS.
- Debug with `nslookup kubernetes.default` from a `busybox:1.28` Pod.

Storage: Volumes, PV, PVC & StorageClasses

Containers are ephemeral — when a container restarts, everything written to its writable layer is lost. Kubernetes **Volumes** decouple storage lifecycle from the container, and the **PersistentVolume** subsystem decouples it further from the Pod, so data outlives Pods and can be reattached elsewhere. This chapter is one of the highest-value Storage topics on the CKA (the Storage domain is ~10% of the exam).

The volume hierarchy at a glance

Object	Scope	Lifecycle	Who creates it
`volume` (in Pod spec)	Pod	Dies with the Pod (except persistent types)	App author
PersistentVolume (PV)	Cluster (not namespaced)	Independent of any Pod	Admin or dynamic provisioner
PersistentVolumeClaim (PVC)	Namespaced	Independent of Pod; binds a PV	App author / user
StorageClass (SC)	Cluster	Template for dynamic PVs	Admin

Ephemeral volumes you must know

`emptyDir` lives as long as the Pod and is shared between its containers — the classic way two containers in a Pod exchange files. `hostPath` mounts a path from the node (dangerous, node-bound — used for system pods and labs).

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: shared-data
spec:
  containers:
    - name: writer
      image: busybox
      command: ["sh","-c","echo hello > /data/msg; sleep 3600"]
      volumeMounts:
        - name: scratch          # must match volume name below
          mountPath: /data
    - name: reader
      image: busybox
      command: ["sh","-c","cat /data/msg; sleep 3600"]
      volumeMounts:
        - name: scratch
          mountPath: /data
  volumes:
    - name: scratch
      emptyDir: {}               # shared, wiped when Pod is removed
```

> 📌 **NOTE** An `emptyDir` with `medium: Memory` is a tmpfs (RAM-backed) — fast, but counts against the container's memory limit.

PersistentVolume (PV) — the supply

A PV is a piece of storage in the cluster, provisioned by an admin or dynamically. Key fields: `capacity`, `accessModes`, `persistentVolumeReclaimPolicy`, and the backend (here `hostPath` for a single-node lab).

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-data
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce            # RWO: one node r/w. Also ROX, RWX, RWOP
  persistentVolumeReclaimPolicy: Retain   # Retain | Delete | (Recycle=deprecated)
  storageClassName: manual     # empty-string "" means "no class"
  hostPath:
    path: /mnt/data
```

Access mode	Short	Meaning
ReadWriteOnce	RWO	Mounted read-write by a single *node*
ReadOnlyMany	ROX	Read-only by many nodes
ReadWriteMany	RWX	Read-write by many nodes (needs NFS/CephFS etc.)
ReadWriteOncePod	RWOP	Read-write by a single *Pod* (1.22+)

PersistentVolumeClaim (PVC) — the demand

A Pod never references a PV directly — it references a PVC, and the control plane **binds** the claim to a PV that satisfies capacity, access mode, and storageClassName.

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-claim
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: manual     # must match a PV (or a StorageClass for dynamic)
  resources:
    requests:
      storage: 500Mi           # binds to pv-data (1Gi ≥ 500Mi requested)
```

```yaml
apiVersion: v1
kind: Pod
metadata: {name: app}
spec:
```

```
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - name: web
          mountPath: /usr/share/nginx/html
  volumes:
    - name: web
      persistentVolumeClaim:
        claimName: data-claim    # Pod → PVC → PV
```

Dynamic provisioning with StorageClasses

Instead of an admin pre-creating PVs, a **StorageClass** lets a PVC trigger on-demand PV creation through a CSI provisioner. The PVC just names the class.

YAML

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
provisioner: ebs.csi.aws.com        # CSI driver
volumeBindingMode: WaitForFirstConsumer   # delay bind until a Pod is scheduled
reclaimPolicy: Delete
allowVolumeExpansion: true
```

> 🎯 **EXAM TIP** `volumeBindingMode: WaitForFirstConsumer` is the exam-favourite answer when a PVC is stuck `Pending` and the storage is topology-aware (zonal). It defers binding until the scheduler picks a node, avoiding zone mismatches.

Essential commands

BASH

```bash
# Inspect
k get pv,pvc -A
k get pvc data-claim -o wide        # STATUS should be Bound
k describe pvc data-claim           # Events explain why it is Pending

# Which SC is the cluster default?
k get sc                            # look for (default) marker

# Expand a volume (SC must allow it)
k edit pvc data-claim               # bump spec.resources.requests.storage

# Delete order matters with Retain policy
k delete pod app
k delete pvc data-claim             # PV goes to 'Released', not reused
```

Reclaim policies & the Released trap

When a PVC is deleted, the PV's `reclaimPolicy` decides its fate: **Delete** removes the backing storage; **Retain** keeps the data but marks the PV `Released` — and a Released PV will **not** auto-bind to a new claim. You must

manually clear `spec.claimRef` to make it `Available` again.

> ⚠️ **GOTCHA** A PVC stuck `Terminating` almost always means a Pod is still using it. Kubernetes protects in-use PVCs via the `kubernetes.io/pvc-protection` finalizer. Delete the consuming Pod first — do **not** force-remove the finalizer.

Exam-day checklist

- **PVC Pending?** → `describe pvc`. Compare `storageClassName`, `accessModes`, and requested size against available PVs / the SC.
- A PV binds to a PVC only if **capacity ≥ request**, access modes match, and storageClassName matches (empty string on both sides also matches).
- Setting `storageClassName: ""` on a PVC disables dynamic provisioning and forces static binding.
- You cannot shrink a volume — only expand (and only if the SC allows it).

RBAC & ServiceAccounts

Role-Based Access Control (RBAC) is how Kubernetes decides **who** can do **what** to **which** resources. Every request that reaches the API server is authenticated (identity) and then authorized — and on modern clusters the authorizer is almost always RBAC. This is one of the most heavily tested Security topics on the CKA: you must be able to read a policy, spot why a request is denied, and build the four objects fast from the imperative commands. RBAC is **purely additive** — there are no "deny" rules; a request is allowed only if some Role/ClusterRole grants it, and denied by default otherwise.

The four objects at a glance

Object	Scope	What it does
Role	Namespaced	A set of permissions *within one namespace*
ClusterRole	Cluster	Permissions cluster-wide, or for cluster-scoped resources (nodes, PVs), or reusable across namespaces
RoleBinding	Namespaced	Grants a Role (or a ClusterRole) to subjects *in one namespace*
ClusterRoleBinding	Cluster	Grants a ClusterRole to subjects *across the whole cluster*

The rule of thumb: the **Role** object holds the permissions; the **Binding** object attaches those permissions to subjects. A binding never adds permissions of its own — it only references a role.

Anatomy of a rule: verbs, resources, apiGroups

A permission rule is a triple. `apiGroups` selects the API group (the core group is the empty string `""`), `resources` names the resource type (plural, lowercase — `pods`, `deployments`), and `verbs` names the allowed actions.

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: dev
  name: pod-reader
rules:
  - apiGroups: [""]                # "" = core group (pods, services, configmaps ... )
    resources: ["pods", "pods/log"] # subresources use a slash
    verbs: ["get", "watch", "list"]
  - apiGroups: ["apps"]            # deployments live in the apps group
    resources: ["deployments"]
    verbs: ["get", "list"]
    resourceNames: ["web"]         # optional: restrict to a named object
```

Verb	HTTP	Meaning
get	GET	Read a single named object
list	GET	List a collection
watch	GET	Stream changes

create	POST	Make a new object
update / patch	PUT / PATCH	Modify an object
delete / deletecollection	DELETE	Remove one / many

🎯 **EXAM TIP** To find the exact `apiGroups` and resource names for a rule, run `k api-resources` — the APIVERSION column gives the group, and the NAME column gives the resource name to put in `resources`. A blank group column means the core group (`""`).

Subjects: User, Group, ServiceAccount

Bindings attach roles to **subjects**. There are three kinds. **Users** and **Groups** are not Kubernetes objects — they are strings that the authentication layer (client certs, OIDC tokens) asserts; the API server never "creates" a user. **ServiceAccounts** *are* real namespaced objects, used by Pods (in-cluster identities).

YAML

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: dev
subjects:
  - kind: User
    name: jane                      # cert CN or OIDC username; no apiGroup prefix issues
    apiGroup: rbac.authorization.k8s.io
  - kind: Group
    name: developers
    apiGroup: rbac.authorization.k8s.io
  - kind: ServiceAccount
    name: build-bot                 # ServiceAccounts have NO apiGroup field
    namespace: dev
roleRef:                            # exactly one role; immutable after creation
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

⚠️ **GOTCHA** `roleRef` is immutable. To point a binding at a different role you must delete and recreate it. Also note the asymmetry: `User` / `Group` subjects carry `apiGroup: rbac.authorization.k8s.io`, but a `ServiceAccount` subject has **no** apiGroup and **requires** a namespace.

The RoleBinding + ClusterRole combo

A powerful pattern: a **RoleBinding** may reference a **ClusterRole**. This grants the ClusterRole's permissions but *only within the RoleBinding's namespace*. This is how the built-in `view`, `edit`, and `admin` ClusterRoles are reused per-namespace without duplicating rules everywhere.

BASH

```bash
# Give user 'jane' the built-in 'view' ClusterRole, but only in namespace dev
k create rolebinding jane-view --clusterrole=view --user=jane -n dev
```

📌 **NOTE** The reverse never works the way people expect: a ClusterRoleBinding to a ClusterRole grants access in **every** namespace *and* to cluster-scoped resources. A RoleBinding can only ever grant within its own namespace, even when it points at a ClusterRole.

ServiceAccounts and how Pods use them

Every Pod runs as a ServiceAccount — if you don't set one, it uses `default` in its namespace. The API server mounts a short-lived, audience-bound **projected token** into the Pod at `/var/run/secrets/kubernetes.io/serviceaccount/`, so code inside the Pod can call the API as that identity.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: api-caller
spec:
  serviceAccountName: build-bot          # use a specific SA, not 'default'
  automountServiceAccountToken: true     # set false to withhold the token entirely
  containers:
    - name: c
      image: curlimages/curl
      command: ["sleep","3600"]
```

Modern clusters (1.24+) no longer auto-create a long-lived Secret token per ServiceAccount. Instead the kubelet requests a **projected ServiceAccount token** via the TokenRequest API — it is time-bound, audience-scoped, and auto-rotated. You can control it explicitly with a projected volume:

```yaml
  volumes:
    - name: token
      projected:
        sources:
          - serviceAccountToken:
              path: token
              expirationSeconds: 3600      # rotated before expiry
              audience: vault               # bind token to a specific audience
```

🎯 **EXAM TIP** Setting `automountServiceAccountToken: false` (on the Pod or on the ServiceAccount object itself) is a common hardening step for workloads that never talk to the API server — no token is mounted, shrinking the blast radius of a compromise.

Aggregated ClusterRoles

A ClusterRole can be built automatically from other ClusterRoles using an `aggregationRule` with label selectors. The controller manager unions the `rules` of every ClusterRole matching the selector. This is exactly how the built-in `admin` / `edit` / `view` roles grow when a CRD ships its own ClusterRole labelled for aggregation.

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: monitoring
aggregationRule:
```

```
  clusterRoleSelectors:
    - matchLabels:
        rbac.example.com/aggregate-to-monitoring: "true"
rules: []      # left empty; the controller fills this in from matching ClusterRoles
```

Imperative creation — fast on the exam

BASH

```
# Role + RoleBinding (namespaced)
k create role pod-reader --verb=get,list,watch --resource=pods -n dev
k create rolebinding read-pods --role=pod-reader --user=jane -n dev

# ClusterRole + ClusterRoleBinding (cluster-wide)
k create clusterrole node-viewer --verb=get,list --resource=nodes
k create clusterrolebinding jane-nodes --clusterrole=node-viewer --user=jane

# Bind a role to a ServiceAccount (note the ns:name form)
k create rolebinding sa-read --role=pod-reader \
  --serviceaccount=dev:build-bot -n dev

# Generate YAML instead of applying — the exam workhorse
k create clusterrole deploy-mgr --verb=create,delete \
  --resource=deployments.apps --dry-run=client -o yaml
```

Testing access: auth can-i and impersonation

`kubectl auth can-i` asks the API server's authorizer directly — the fastest way to verify a policy without switching credentials. Combine it with `--as` (user impersonation) and `--as-group` to test *another* identity's access, which itself requires the `impersonate` verb.

BASH

```
# Can I do it?
k auth can-i create deployments -n dev              # → yes / no
k auth can-i '*' '*'                                 # am I a cluster-admin?
k auth can-i list secrets -n dev --list             # show everything I can do

# Test someone else's access via impersonation
k auth can-i get pods -n dev --as=jane
k auth can-i get pods -n dev --as=system:serviceaccount:dev:build-bot
k get pods -n dev --as=jane --as-group=developers    # run a real request as jane
```

🎯 **EXAM TIP** The ServiceAccount username format is `system:serviceaccount:<namespace>:<name>`, and all SAs also belong to the group `system:serviceaccounts`. Memorize this string — questions often ask you to grant or test access for a specific Pod's identity.

Exam-day checklist

- **Access denied?** Trace it: identity → is there a Role/ClusterRole granting the verb+resource+group? → is there a binding attaching it to that subject *in the right namespace*? Use `k auth can-i ... --as=<subject>` to confirm.

- Core resources (pods, services, secrets) are group `""`; deployments/ replicasets/daemonsets are group `apps`; roles/bindings are `rbac.authorization.k8s.io`.
- Cluster-scoped resources (nodes, PVs, namespaces) can **only** be granted via a ClusterRole — a namespaced Role cannot cover them.
- `roleRef` is immutable; recreate the binding to change the referenced role.
- Subresources like `pods/log`, `pods/exec` are separate resources and need their own rule.

Security Contexts

A **securityContext** defines the privilege and access-control settings for a Pod or container — which UID/GID the process runs as, whether it can escalate privileges, which Linux capabilities it holds, whether its root filesystem is writable, and which seccomp profile confines its syscalls. These settings map directly onto Linux kernel primitives (user namespaces, capabilities, no_new_privs, seccomp-bpf), so understanding them means understanding *why* a container is or isn't allowed to do something. On the CKA you should be able to make a container run as a non-root user, drop capabilities, and read the resulting effective settings quickly.

Two levels: Pod vs container

There are two `securityContext` blocks. The **Pod-level** one (`spec.securityContext`) sets defaults that apply to *all* containers and governs volume ownership. The **container-level** one (`spec.containers[].securityContext`) applies to a single container and **overrides** the Pod-level value where the two overlap. Not all fields exist at both levels — `fsGroup` is Pod-only; `capabilities`, `privileged`, `readOnlyRootFilesystem`, and `allowPrivilegeEscalation` are container-only.

Field	Level	Effect
runAsUser / runAsGroup	Both	UID/GID the process runs as
runAsNonRoot	Both	Refuse to start if the image would run as UID 0
fsGroup	Pod	Supplemental GID that owns mounted volumes
allowPrivilegeEscalation	Container	Controls the no_new_privs bit
privileged	Container	Full host access — effectively root on node
readOnlyRootFilesystem	Container	Mount `/` read-only
capabilities	Container	add / drop Linux capabilities
seccompProfile	Both	Syscall filter profile

Running as non-root

`runAsNonRoot: true` is a guardrail, not an assignment: it does not choose a UID, it just makes the kubelet **refuse to start** the container if it would run as UID 0. Pair it with an explicit `runAsUser` to both pick an identity and enforce the rule.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: hardened
spec:
  securityContext:              # Pod level: defaults + volume ownership
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000               # mounted volumes get group 2000, group-writable
    seccompProfile:
      type: RuntimeDefault      # use the container runtime's default seccomp filter
  containers:
```

```
    - name: app
      image: busybox
      command: ["sh","-c","id; sleep 3600"]
      securityContext:           # container level: overrides + container-only fields
        runAsNonRoot: true
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop: ["ALL"]          # drop everything, then add back only what's needed
          add: ["NET_BIND_SERVICE"]  # e.g. bind to ports < 1024 without being root
```

📌 **NOTE** `fsGroup` makes the kubelet `chown` /set the SGID on volume contents to that GID so a non-root process can write to a PersistentVolume. Without it, a container running as UID 1000 often hits "permission denied" on a freshly-provisioned volume. Large volumes can be slow to relabel — `fsGroupChangePolicy: OnRootMismatch` skips the walk when ownership already looks correct.

allowPrivilegeEscalation and privileged

`allowPrivilegeEscalation: false` sets the kernel `no_new_privs` flag, preventing a process from gaining more privileges than its parent — this blocks setuid binaries (like `sudo`) from elevating. It is one of the cheapest, highest-value hardening settings. `privileged: true` is the opposite extreme: it grants nearly all capabilities and access to host devices, effectively giving the container root on the node. Reserve it for genuine system agents (CNI, storage drivers).

⚠️ **GOTCHA** `privileged: true` implicitly forces `allowPrivilegeEscalation: true` and hands the container almost every capability regardless of your `drop` list. Never combine "privileged" with an untrusted workload — a breakout gives the attacker the node.

Linux capabilities

Instead of the all-or-nothing root/non-root split, Linux breaks root's powers into **capabilities** (e.g. `NET_ADMIN`, `SYS_TIME`, `NET_BIND_SERVICE`). Best practice: `drop: ["ALL"]` then add back only the specific ones the app needs. In YAML you write capability names **without** the `CAP_` prefix (`NET_BIND_SERVICE`, not `CAP_NET_BIND_SERVICE`).

BASH

```
# Verify what a running container actually got
k exec hardened -- id                        # uid=1000 gid=3000 groups=2000
k exec hardened -- cat /proc/1/status | grep Cap   # CapEff bitmask of effective caps
k exec hardened -- touch /test               # should fail with read-only root fs
```

seccomp

seccomp ("secure computing") filters the syscalls a process may make. `type: RuntimeDefault` applies the container runtime's curated allowlist and is a strong, near-free default. `type: Localhost` uses a custom profile file on the node under the kubelet's seccomp root; `type: Unconfined` disables filtering (avoid). Setting `RuntimeDefault` Pod-wide is a common recommendation.

YAML

```
    securityContext:
      seccompProfile:
```

```
          type: Localhost
          localhostProfile: profiles/audit.json   # relative to the kubelet seccomp dir
```

Pod Security Admission (PSA)

PodSecurityPolicy is gone (removed in 1.25). Its replacement is **Pod Security Admission**, a built-in admission controller that enforces the three **Pod Security Standards** at the *namespace* level via labels — no extra objects to create.

Standard	Meaning
`privileged`	Unrestricted — no constraints at all
`baseline`	Blocks known privilege escalations (no hostNetwork, no privileged, etc.); minimally restrictive
`restricted`	Heavily hardened — requires runAsNonRoot, drop ALL caps, seccomp RuntimeDefault, no privilege escalation

Each namespace can request three **modes** per standard: `enforce` (reject violating Pods), `audit` (allow but log), and `warn` (allow but return a client warning). You set them as labels:

BASH
```
k label ns prod \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/warn=restricted \
  pod-security.kubernetes.io/audit=restricted
```

YAML
```
apiVersion: v1
kind: Namespace
metadata:
  name: prod
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: latest   # pin to a version if desired
    pod-security.kubernetes.io/warn: restricted
```

> 🎯 **EXAM TIP** If a Pod is rejected with a message about "violates PodSecurity", check the namespace's `pod-security.kubernetes.io/enforce` label. The fix is either to harden the Pod's securityContext to meet the standard (typical for `restricted`) or, if permitted, to relax the namespace label.

Exam-day checklist

- Container-level securityContext **overrides** Pod-level where both set the same field. Remember `fsGroup` is Pod-only; `capabilities`, `privileged`, `readOnlyRootFilesystem` are container-only.
- Verify results with `k exec <pod> -- id` — don't assume the YAML took effect.
- `runAsNonRoot: true` only *enforces*; it does not pick a UID. Add `runAsUser` to choose one.
- Capabilities are written without the `CAP_` prefix; the hardened pattern is `drop: ["ALL"]` then `add` the minimum.
- PSA is namespace-label driven with modes enforce/audit/warn and standards privileged/baseline/restricted — no PSP objects exist anymore.

Cluster PKI, TLS & Certificates

Every component in a Kubernetes control plane talks to every other over mutual TLS. The API server presents a serving certificate; the kubelet, scheduler, controller-manager, and even you (via kubeconfig) present client certificates that the API server verifies against a trusted **Certificate Authority (CA)**. Understanding this public-key infrastructure (PKI) is essential for the CKA: certificate expiry, a wrong CA, or a bad kubeconfig are classic "cluster is broken" scenarios you must diagnose from the node.

The cluster CA and where certs live

A `kubeadm` cluster generates a self-signed root CA and issues every component's certificate from it. All of this material lives on the control-plane node under `/etc/kubernetes/pki`. etcd has its own sub-CA in `pki/etcd`.

```bash
ls -l /etc/kubernetes/pki
# ca.crt  ca.key                 ← the cluster root CA (signs client & serving certs)
# apiserver.crt  apiserver.key   ← API server serving cert
# apiserver-kubelet-client.crt   ← API server → kubelet client cert
# front-proxy-ca.crt/.key        ← aggregation layer CA
# sa.key  sa.pub                  ← ServiceAccount token signing keys (NOT an x509 cert)
# etcd/ca.crt  etcd/server.crt  etcd/peer.crt   ← etcd PKI
```

Certificate	Role	Signed by
apiserver.crt	API server serving cert (clients verify it)	cluster CA
apiserver-kubelet-client.crt	API server acting as client to kubelets	cluster CA
kubelet client cert	kubelet authenticating to API server	cluster CA (or auto-issued)
etcd/server.crt, peer.crt	etcd serving & peer communication	etcd CA
front-proxy-client.crt	Aggregation layer / extension apiservers	front-proxy CA

> 📌 **NOTE** The kubelet is special: its own serving/client certs are usually stored under `/var/lib/kubelet/pki/` (not `/etc/kubernetes/pki`), and the kubelet can rotate its client cert automatically via the CSR API when `rotateCertificates` is enabled.

Reading a certificate

To answer "what's the expiry, who signed it, what SANs does it cover?" decode the cert with OpenSSL. The `-text -noout` combination is the one to memorize.

```bash
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -text -noout

# Look for:
#   Issuer: CN = kubernetes                ← signing CA
#   Subject: CN = kube-apiserver           ← identity (CN = username for client certs)
#   Validity  Not Before / Not After       ← expiry window
#   X509v3 Subject Alternative Name         ← DNS/IP SANs the serving cert is valid for
```

🎯 **EXAM TIP** For a client certificate, the **Subject CN** is the username the API server authenticates you as, and the **Organization (O)** fields become your groups. This is exactly how `kubeadm` 's admin cert works: CN `kubernetes-admin` , O `kubeadm:cluster-admins` .

kubeadm certs: checking and renewing expiry

kubeadm-managed certs are valid for one year and are renewed automatically on control plane upgrade — but if you skip upgrades, they expire and the API server stops trusting its clients. Two commands handle the whole lifecycle.

```bash
# See every cert's expiry at a glance
kubeadm certs check-expiration

# Renew everything (or a single cert by name)
kubeadm certs renew all
kubeadm certs renew apiserver

# After renewal, restart the static-pod control plane so they reload the new certs
# (moving the manifest out and back, or restarting kubelet, forces a reload)
```

⚠️ **GOTCHA** Certificate renewal does **not** update your `admin.conf` / kubeconfig if it embeds a client cert — `kubeadm certs renew all` covers the kubeconfig files under `/etc/kubernetes/*.conf` , but a copied `~/.kube/config` keeps the old embedded cert. Re-copy `/etc/kubernetes/admin.conf` after renewing.

The CertificateSigningRequest (CSR) API workflow

To onboard a new user, you don't hand-edit files on the API server — you use the **CertificateSigningRequest** API. The user generates a private key and a CSR, submits the CSR to the cluster, an admin approves it, and the cluster CA signs and returns a certificate. This is a favourite CKA task.

Step 1 — user creates a key and a CSR (the CN becomes the username, O becomes the group):

```bash
openssl genrsa -out jane.key 2048
openssl req -new -key jane.key -out jane.csr \
  -subj "/CN=jane/O=developers"
```

Step 2 — submit a CertificateSigningRequest object. The `request` field is the base64 of the PEM CSR (no line wraps):

```yaml
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: jane
spec:
  request: LS0tLS1CRUdJTi ...            # base64 -w0 of jane.csr
  signerName: kubernetes.io/kube-apiserver-client   # signer for client-auth certs
  expirationSeconds: 604800              # 7 days (optional)
  usages:
    - client auth
```

```bash
# Build the request value cleanly and create the object
cat jane.csr | base64 -w0        # paste into spec.request, or:
k create -f jane-csr.yaml
```

Step 3 — admin approves, then retrieve the signed cert:

```bash
k get csr                              # STATUS: Pending
k certificate approve jane             # or: k certificate deny jane
k get csr jane -o jsonpath='{.status.certificate}' | base64 -d > jane.crt
```

> 🎯 **EXAM TIP** The `signerName` matters. Use `kubernetes.io/kube-apiserver-client` for a user/client cert (with usage `client auth`). `kubernetes.io/kubelet-serving` is for kubelet serving certs. Choosing the wrong signer is a common reason an approved CSR yields an unusable cert.

Turning a cert into cluster access

Once `jane` has `jane.crt` and `jane.key`, wire them into a kubeconfig. Remember: authentication only proves identity — Jane still needs RBAC bindings to actually *do* anything.

```bash
k config set-credentials jane \
  --client-certificate=jane.crt --client-key=jane.key --embed-certs=true
k config set-context jane@cluster --cluster=kubernetes --user=jane
k config use-context jane@cluster
k auth can-i get pods                  # 'no' until an RBAC binding exists for jane
```

kubeconfig structure

A kubeconfig is a YAML file with three lists plus a pointer. **clusters** hold the API server URL and its CA to trust; **users** hold credentials (client cert/key, token, or exec plugin); **contexts** pair a cluster + user + default namespace; and `current-context` selects the active one.

```yaml
apiVersion: v1
kind: Config
current-context: jane@cluster
clusters:
  - name: kubernetes
    cluster:
      server: https://10.0.0.1:6443
      certificate-authority-data: LS0tLS1 ...    # CA the client trusts (base64 ca.crt)
users:
  - name: jane
    user:
      client-certificate-data: LS0t ...          # base64 jane.crt
      client-key-data: LS0t ...                  # base64 jane.key
contexts:
  - name: jane@cluster
    context:
      cluster: kubernetes
      user: jane
      namespace: dev                             # default ns for this context
```

```bash
k config view                          # merged view (keys redacted)
k config get-contexts                  # * marks current
k config use-context jane@cluster      # switch
k config set-context --current --namespace=dev   # change default ns without editing YAML
```

> 📌 **NOTE** `kubectl` resolves its config from `--kubeconfig`, then the `$KUBECONFIG` env var (colon/semicolon-separated list, merged in order), then `~/.kube/config`. On CKA nodes the admin file is `/etc/kubernetes/admin.conf`.

Exam-day checklist

- Control-plane certs live in `/etc/kubernetes/pki`; kubelet certs in `/var/lib/kubelet/pki`. `ca.crt` is the trust anchor for the whole cluster.
- Diagnose expiry with `kubeadm certs check-expiration`; fix with `kubeadm certs renew all` then restart the control-plane static pods.
- Decode any cert with `openssl x509 -in <file> -text -noout`; the client CN = username, O = group.
- CSR flow: `genrsa` + `req` → base64 into a CSR object with `signerName: kubernetes.io/kube-apiserver-client` → `k certificate approve` → extract `.status.certificate` and base64-decode.
- A valid cert authenticates but authorizes nothing — pair every new user with an RBAC Role/RoleBinding.

PART III

Cluster Administration & Troubleshooting

The admin half of the CKA — where most of the marks are. Building and upgrading clusters with kubeadm, backing up etcd, static pods, node maintenance, and the systematic troubleshooting workflow that separates a pass from a fail.

kubeadm Cluster Installation

kubeadm is the official tool for bootstrapping a conformant Kubernetes cluster. It does not install the container runtime, a CNI plugin, or the CLI tools for you — it wires up the control-plane components (API server, controller-manager, scheduler, etcd) as static Pods and issues the certificates that make them trust each other. On the CKA you will be asked to *join* a node or repair a broken bootstrap far more often than to build from zero, but you must know the full sequence cold. Every step below runs as `root` (or with `sudo`).

Step 0 — prerequisites on EVERY node

These must be done on the control-plane node **and** every worker. The kubelet will refuse to start if swap is on (unless explicitly configured), and pod networking breaks without the bridge netfilter module and IP forwarding.

```bash
# 1. Disable swap (kubelet requirement)
sudo swapoff -a
# make it permanent: comment out the swap line in /etc/fstab
sudo sed -i '/ swap / s/^/#/' /etc/fstab

# 2. Load kernel modules needed by the CNI / kube-proxy
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
sudo modprobe overlay
sudo modprobe br_netfilter

# 3. sysctl so bridged traffic is seen by iptables + enable forwarding
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
sudo sysctl --system            # apply without reboot
```

⚠️ **GOTCHA** If `br_netfilter` is not loaded, the two `bridge-nf-call-*` sysctl keys will not even exist and `sysctl --system` silently skips them — Pods then can't reach each other. Always `modprobe` first.

Step 1 — install a container runtime (containerd)

Kubernetes talks to the runtime over CRI. containerd is the common choice. The single most common bootstrap failure is leaving containerd's default config, which sets `SystemdCgroup = false` while the kubelet expects the systemd cgroup driver.

```bash
sudo apt-get update && sudo apt-get install -y containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml >/dev/null

# Flip the cgroup driver to systemd
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
```

```
sudo systemctl restart containerd
sudo systemctl enable containerd
```

Step 2 — install kubeadm, kubelet, kubectl

Install the same minor version on every node. The kubelet is enabled but will crash-loop until `kubeadm init` / `join` writes its config — that is expected.

BASH

```
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg

# Repo key + list for the desired minor line (e.g. v1.30)
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key \
  | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
  https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /' \
  | sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl   # pin so apt upgrade won't bump them
sudo systemctl enable --now kubelet
```

> 🎯 **EXAM TIP** `apt-mark hold` is important: an accidental `apt upgrade` that jumps kubeadm two minor versions will make the upgrade chapter's procedure impossible. You `unhold` deliberately only when upgrading.

Step 3 — initialize the control plane

Run only on the first control-plane node. `--pod-network-cidr` must match what your CNI expects (Calico's default is `192.168.0.0/16`; Flannel uses `10.244.0.0/16`). This is the flag examinees most often omit.

BASH

```
sudo kubeadm init --pod-network-cidr=192.168.0.0/16 \
  --apiserver-advertise-address=<CONTROL-PLANE-IP>
```

On success kubeadm prints two things you must keep: the `kubectl` setup lines, and a `kubeadm join ...` command for workers. Set up kubeconfig for your normal user immediately:

BASH

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
# (root can instead: export KUBECONFIG=/etc/kubernetes/admin.conf)

kubectl get nodes    # control-plane shows NotReady until a CNI is installed
```

Step 4 — install a CNI (Pod network)

The node stays `NotReady` and `coredns` Pods stay `Pending` until a CNI is applied. Apply it once, from the control plane:

```bash
# Example: Calico
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.27.0/manifests/calico.ya

kubectl get pods -n kube-system -w   # wait for calico + coredns to go Running
kubectl get nodes                    # control-plane → Ready
```

Step 5 — join worker nodes

Run the join command kubeadm printed, on each worker (with `sudo`). It contains the API endpoint, a bootstrap token, and the CA cert hash:

```bash
sudo kubeadm join 10.0.0.10:6443 --token abcdef.0123456789abcdef \
  --discovery-token-ca-cert-hash sha256:<hash>
```

Bootstrap tokens expire after **24 hours** by default. When you lost the original command or it expired, regenerate it from the control plane — this is a very common exam step:

```bash
# Prints a complete, ready-to-paste join command with a fresh token
kubeadm token create --print-join-command

# See existing tokens / TTLs
kubeadm token list

# Reconstruct just the CA cert hash if you have the token but not the hash
openssl x509 -pubkey -in /etc/kubernetes/pki/ca.crt \
  | openssl rsa -pubin -outform der 2>/dev/null \
  | openssl dgst -sha256 -hex | sed 's/^.* //'
```

Verify from the control plane. Workers show role `<none>` — that is normal, kubeadm does not label them:

```bash
kubectl get nodes -o wide
# optional cosmetic label:
kubectl label node worker-1 node-role.kubernetes.io/worker=worker
```

Troubleshooting a failed init/join

Symptom	Likely cause / fix
Preflight error: swap	`swapoff -a` not run on that node
kubelet not running / CrashLoop	cgroup driver mismatch — set `SystemdCgroup = true`, restart containerd
Node stuck NotReady	No CNI installed, or Pod CIDR mismatch
join: token expired	`kubeadm token create --print-join-command`
Need to retry init/join	`sudo kubeadm reset -f` then clean `~/.kube` and CNI iptables/CNI config

> 📌 **NOTE** `kubeadm reset -f` tears down the local state so you can re-run `init` or `join`. Afterwards remove `$HOME/.kube/config` and any leftover `/etc/cni/net.d` files before retrying.

Exam-day checklist

- Prereqs run on **every** node: swap off, `br_netfilter` + sysctl, runtime with systemd cgroups.
- `kubeadm init` only on the first CP; **match** `--pod-network-cidr` to your CNI.
- Immediately copy `/etc/kubernetes/admin.conf` to `~/.kube/config`.
- Node NotReady after init = you still need to install a CNI.
- Lost/expired join command → `kubeadm token create --print-join-command`.

Cluster Upgrades with kubeadm

Upgrading a kubeadm cluster is one of the most reliable point-earners on the CKA — and one of the easiest to fail by doing the steps out of order. The rules are simple but absolute: upgrade **the control plane first**, then the workers; move only **one minor version at a time** (1.29 → 1.30, never 1.29 → 1.31); and within each node the `kubeadm` binary is upgraded *before* you run any `kubeadm upgrade` subcommand. All version numbers below use v1.30.x as the target — read the exam question for the exact patch version and substitute it everywhere.

> ⚠️ **GOTCHA** Skipping minor versions is unsupported. If the cluster is on 1.28 and the task wants 1.30, you must upgrade 1.28→1.29 fully, then 1.29→1.30. Also confirm your OS package repo points at the right minor line before `apt-get update`.

Order of operations (memorize this)

#	Where	Action
1	Control plane	upgrade `kubeadm`, then `kubeadm upgrade apply`
2	Control plane	upgrade `kubelet` + `kubectl`, restart kubelet
3	Each worker	drain, upgrade `kubeadm`, `kubeadm upgrade node`, upgrade kubelet, restart, uncordon

Part A — the control-plane node

```bash
# 1. Point the package repo at the new minor line if needed, then refresh
sudo sed -i 's#v1.29#v1.30#' /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update

# 2. See exactly which kubeadm patch versions are installable
apt-cache madison kubeadm      # note the full string, e.g. 1.30.2-1.1

# 3. Drain the control-plane node (evict workloads, keep DaemonSets)
kubectl drain <cp-node> --ignore-daemonsets --delete-emptydir-data

# 4. Unhold and upgrade ONLY the kubeadm binary first
sudo apt-mark unhold kubeadm
sudo apt-get install -y kubeadm=1.30.2-1.1
sudo apt-mark hold kubeadm

# 5. Verify kubeadm sees the new version
kubeadm version

# 6. Dry-run: shows the upgrade path and any component versions
sudo kubeadm upgrade plan

# 7. Apply the upgrade to the control-plane components (etcd, apiserver, etc.)
sudo kubeadm upgrade apply v1.30.2
```

> 📌 **NOTE** `kubeadm upgrade apply` upgrades the **static-Pod control plane and etcd** and rotates certs — it does **not** touch the kubelet. That is why kubelet is a separate step. On additional control-plane nodes you use `kubeadm`

`upgrade node` instead of `apply`.

```bash
# 8. Now upgrade the kubelet and kubectl on this node
sudo apt-mark unhold kubelet kubectl
sudo apt-get install -y kubelet=1.30.2-1.1 kubectl=1.30.2-1.1
sudo apt-mark hold kubelet kubectl

# 9. Reload systemd and restart the kubelet to pick up the new binary
sudo systemctl daemon-reload
sudo systemctl restart kubelet

# 10. Bring the node back into scheduling
kubectl uncordon <cp-node>

# Verify
kubectl get nodes    # cp node should show the new VERSION and be Ready
```

Part B — each worker node

Repeat for every worker, ideally one at a time. Draining is issued **from the control plane** (where your kubeconfig lives); the remaining upgrade commands run **on the worker itself** over SSH.

```bash
# --- run from the control plane ---
kubectl drain <worker> --ignore-daemonsets --delete-emptydir-data

# --- SSH to the worker, then run there ---
sudo sed -i 's#v1.29#v1.30#' /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update

sudo apt-mark unhold kubeadm
sudo apt-get install -y kubeadm=1.30.2-1.1
sudo apt-mark hold kubeadm

# Upgrade the LOCAL kubelet config for this node (NOT 'upgrade apply')
sudo kubeadm upgrade node

sudo apt-mark unhold kubelet kubectl
sudo apt-get install -y kubelet=1.30.2-1.1 kubectl=1.30.2-1.1
sudo apt-mark hold kubelet kubectl

sudo systemctl daemon-reload
sudo systemctl restart kubelet

# --- back on the control plane ---
kubectl uncordon <worker>
kubectl get nodes
```

⚠️ **GOTCHA** Workers use `kubeadm upgrade node` — never `kubeadm upgrade apply`. Running `apply` on a worker (or on a second control-plane node) is a classic mistake.

Why drain, and the flags that matter

`kubectl drain` cordons the node (marks it unschedulable) and evicts its Pods so the upgrade can restart the kubelet without disrupting running work unexpectedly. Two flags are almost always required in the exam environment:

- `--ignore-daemonsets` — DaemonSet Pods can't be rescheduled elsewhere, so drain refuses to proceed without this. Nearly every cluster runs kube-proxy/CNI as DaemonSets.
- `--delete-emptydir-data` — required if any Pod uses an `emptyDir`, otherwise drain aborts to protect that data.
- `--force` — needed if there are unmanaged (bare) Pods not backed by a controller.

```bash
kubectl drain node01 --ignore-daemonsets --delete-emptydir-data --force
# ... after upgrade ...
kubectl uncordon node01     # do NOT forget this — the node stays unschedulable otherwise
```

Verifying and common failures

```bash
kubectl get nodes                # every node shows the target VERSION + Ready
kubeadm upgrade plan             # should report you are on the latest of that line
kubectl get pods -n kube-system  # control-plane static pods Running with new image tags
```

Symptom	Cause / fix
`kubeadm upgrade plan` can't find the version	Repo still on old minor line — fix the `kubernetes.list` and `apt-get update`
apt won't install new version	Package still `hold` — `apt-mark unhold` first
Node still shows old VERSION	Forgot `systemctl restart kubelet` after installing the new kubelet
Node still SchedulingDisabled	Forgot `kubectl uncordon`
Pods won't evict on drain	Add `--ignore-daemonsets --delete-emptydir-data` (and `--force` for bare Pods)

Exam-day checklist

- **Control plane first**, workers after. One minor version per pass.
- Upgrade the `kubeadm` binary **before** any `kubeadm upgrade` command.
- CP node uses `kubeadm upgrade apply vX.Y.Z`; workers use `kubeadm upgrade node`.
- kubelet/kubectl are upgraded separately, then `daemon-reload` + `restart kubelet`.
- Always finish a node with `kubectl uncordon`.

etcd Backup & Restore

etcd is the cluster's single source of truth — a distributed key-value store that holds the entire Kubernetes object graph: every Pod, Secret, ConfigMap, RBAC rule, and the current state written by controllers. Lose etcd and you lose the cluster; nothing outside etcd (except the container images and static-Pod manifests on disk) is authoritative. That is why backing up and restoring etcd is an **extremely common** CKA task, and one where tiny mistakes in flags or paths cost the whole question. Get the certificate paths and the `ETCDCTL_API=3` environment exactly right.

How etcd runs in a kubeadm cluster

kubeadm runs etcd as a **static Pod** on the control-plane node, defined by `/etc/kubernetes/manifests/etcd.yaml`. Its data lives in `/var/lib/etcd`, and its TLS material is under `/etc/kubernetes/pki/etcd/`. You interact with it using `etcdctl`, which must be told to speak the v3 API.

```bash
export ETCDCTL_API=3        # v2 is the legacy default on old builds; always set 3
etcdctl version             # confirm the client is present
```

> 🎯 **EXAM TIP** If `etcdctl` is not installed on the node, you can run it inside the etcd container: `kubectl -n kube-system exec -it etcd-<node> -- sh`, or `apt-get install -y etcd-client`. On the exam it is usually already on the control-plane node.

Finding the certificate paths (don't guess)

The exact cert/key/CA paths and the listen endpoint are declared as flags in the etcd static-Pod manifest. Read them from there rather than memorizing — the manifest never lies:

```bash
grep -E 'cert-file|key-file|trusted-ca-file|listen-client-urls' \
  /etc/kubernetes/manifests/etcd.yaml
```

On a standard kubeadm cluster this yields:

etcdctl flag	Value from manifest
`--cacert`	`/etc/kubernetes/pki/etcd/ca.crt`
`--cert`	`/etc/kubernetes/pki/etcd/server.crt`
`--key`	`/etc/kubernetes/pki/etcd/server.key`
`--endpoints`	`https://127.0.0.1:2379`

Taking a snapshot (backup)

All four flags are mandatory — etcd requires client-cert authentication, so a snapshot without the certs fails with a context-deadline/connection error.

```bash
ETCDCTL_API=3 etcdctl snapshot save /opt/etcd-backup.db \
  --endpoints=https://127.0.0.1:2379 \
```

```
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
```

Verify the snapshot's integrity and metadata. The status table shows the hash, revision, and total keys — proof the backup is real:

```bash
ETCDCTL_API=3 etcdctl snapshot status /opt/etcd-backup.db --write-out=table
```

📌 **NOTE** The `--endpoints` flag can be omitted for `snapshot save` in some etcd builds (it defaults to localhost), but always include it on the exam so behavior is deterministic. It is **not** used at all by `snapshot restore`, which reads a file, not a live server.

Restoring from a snapshot

Restore never overwrites the live data directory in place. You restore into a **new** directory, then re-point etcd at it by editing the static-Pod manifest. The clean sequence:

```bash
# 1. Restore the snapshot into a fresh data directory
ETCDCTL_API=3 etcdctl snapshot restore /opt/etcd-backup.db \
  --data-dir=/var/lib/etcd-restore
```

`snapshot restore` does **not** need the cert flags — it operates purely on the file. Now tell the etcd static Pod to use the new directory by editing `/etc/kubernetes/manifests/etcd.yaml`. Two things must change to match `/var/lib/etcd-restore`:

```yaml
  # --- 1. the volume hostPath that backs etcd data ---
  volumes:
    - name: etcd-data
      hostPath:
        path: /var/lib/etcd-restore      # was /var/lib/etcd
        type: DirectoryOrCreate

  # --- 2. (only if present) the --data-dir flag on the etcd command ---
  #     spec.containers[0].command:
  #       - --data-dir=/var/lib/etcd-restore
```

🎯 **EXAM TIP** Keep it simple on the exam: the safest edit is to change the `hostPath.path` of the etcd data volume to the restore directory. Because the container's `--data-dir` mounts that volume at `/var/lib/etcd` inside the container, you often only need to change the hostPath, not the command flag. If the question restores into `/var/lib/etcd-from-backup`, use that exact path.

Because etcd is a static Pod, you do **not** apply anything. The kubelet watches the manifests directory and will automatically recreate the etcd Pod when the file changes:

```bash
# kubelet detects the manifest change and restarts etcd automatically.
# You can nudge it if needed:
sudo systemctl restart kubelet
```

```
# Watch etcd (and then the whole control plane) come back
sudo crictl ps | grep etcd
kubectl get pods -n kube-system
kubectl get nodes            # cluster reflects the restored state
```

⚠️ **GOTCHA** Set correct ownership on the restored directory if etcd fails to start: `sudo chown -R etcd:etcd /var/lib/etcd-restore` (matching the user etcd runs as). Also ensure no two data dirs are mounted at once — the API server may briefly go down during the restart; that is expected while etcd reloads.

One-shot patterns worth memorizing

BASH

```
# Backup one-liner (paths are stable on kubeadm)
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /opt/snapshot.db

# Check any cluster member's health (uses same auth flags)
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key endpoint health
```

Exam-day checklist

- Always `export ETCDCTL_API=3` (or prefix each command).
- `snapshot save` needs `--endpoints --cacert --cert --key`; get paths from `etcd.yaml`.
- `snapshot restore` needs only `--data-dir` — restore to a NEW directory.
- Re-point the etcd static Pod: edit the `hostPath` (and `--data-dir` if set) in `/etc/kubernetes/manifests/etcd.yaml`.
- Do not `kubectl apply` — the kubelet restarts the static Pod on manifest change.
- Fix directory ownership if etcd won't start after restore.

Static Pods

A **static Pod** is a Pod managed directly by the **kubelet** on a specific node, not by the API server or the scheduler. The kubelet watches a directory of manifest files and runs whatever Pod specs it finds there, restarting them if they die. Static Pods are how kubeadm bootstraps the control plane itself — the API server, controller-manager, scheduler, and etcd all run as static Pods — so understanding them is essential for both the "run a Pod on a node" tasks and for repairing a broken control plane on the CKA.

What makes a Pod "static"

	Static Pod	Normal (API) Pod
Created by	kubelet, from a file on disk	API server, via scheduler
Scheduler involved?	No — always runs on that node	Yes
Editable with `kubectl`?	No (read-only mirror)	Yes
Survives API server outage?	Yes	Existing ones keep running, but no new scheduling
Controllers (Deployment/RS)?	Cannot manage them	Yes

The staticPodPath and how the kubelet finds manifests

The kubelet reads its own configuration from `/var/lib/kubelet/config.yaml`. The field `staticPodPath` names the directory it scans; on kubeadm clusters this is `/etc/kubernetes/manifests`.

```bash
# Find the configured static-pod directory
grep staticPodPath /var/lib/kubelet/config.yaml
# → staticPodPath: /etc/kubernetes/manifests

# See what's already there (the control plane!)
ls /etc/kubernetes/manifests
# etcd.yaml  kube-apiserver.yaml  kube-controller-manager.yaml  kube-scheduler.yaml
```

Any valid Pod manifest dropped into that directory is started within a second or two; delete the file and the kubelet stops and removes the Pod. There is no `kubectl apply` and no scheduler involved.

> 📌 **NOTE** The path is set by the kubelet's config file, referenced from `/etc/systemd/system/kubelet.service.d/10-kubeadm.conf`. Older setups used the `--pod-manifest-path` flag instead. If you must find it on an unfamiliar node, inspect the running kubelet: `ps aux | grep kubelet` and look for `--config`, then read `staticPodPath` from that file.

Creating a static Pod

Generate the YAML with `kubectl run --dry-run`, then place it in the directory. The `nodeName` is implicit — it is whatever node's directory you write to.

```bash
# Generate a manifest without contacting the API server
kubectl run web --image=nginx --dry-run=client -o yaml > web.yaml
```

```
# Move it into the static-pod directory (as root, on the target node)
sudo mv web.yaml /etc/kubernetes/manifests/web.yaml
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web
spec:
  containers:
    - name: web
      image: nginx
```

Mirror Pods — how static Pods show up in the API

So that you can still *see* static Pods with `kubectl`, the kubelet creates a read-only **mirror Pod** in the API server. Its name is the manifest's Pod name with the node name appended: `<pod-name>-<nodename>`.

```bash
kubectl get pods -o wide
# NAME              READY   STATUS    NODE
# web-node01        1/1     Running   node01     ← mirror of the static Pod

# You cannot edit or delete it via the API:
kubectl delete pod web-node01
# it disappears briefly, then the kubelet RECREATES the mirror from the file
```

> ⚠️ **GOTCHA** To truly delete or change a static Pod you must edit or remove its **manifest file** on the node — `kubectl delete` only removes the mirror, which the kubelet immediately recreates. This trips people up constantly.

You can also confirm the control-plane components are static: their mirror Pods carry the node suffix, e.g. `kube-apiserver-cp01`, `etcd-cp01`.

```bash
kubectl get pods -n kube-system
# etcd-cp01                      1/1  Running   ← static
# kube-apiserver-cp01            1/1  Running   ← static
# kube-controller-manager-cp01   1/1  Running   ← static
# kube-scheduler-cp01            1/1  Running   ← static
```

Why the control plane uses static Pods

There is a bootstrapping chicken-and-egg problem: the API server can't schedule itself before it exists. Static Pods solve it — the kubelet can run the API server from a file with no cluster running yet. This also means you edit control-plane flags by editing the manifests directly (e.g. adding an `--audit-log-path` to `kube-apiserver.yaml`); the kubelet restarts the component on save. If you make a typo, the component won't come back — check with `crictl`:

```bash
sudo crictl ps                  # running containers (works even if API is down)
sudo crictl logs <container-id>   # why did apiserver fail to start?
```

> 🎯 **EXAM TIP** Editing `kube-apiserver.yaml` briefly takes the API server down while it restarts — `kubectl` will hang for a few seconds. That is normal. If it never comes back, you almost certainly have a YAML/flag error in the manifest.

Troubleshooting

Symptom	Cause / fix
Static Pod not appearing	File not in `staticPodPath`, or invalid YAML — check `journalctl -u kubelet`
Deleted with kubectl, it came back	Expected — remove the manifest file instead
Wrong node	Static Pods run only on the node whose directory holds the file; move the file to the intended node
API server down after edit	Bad flag in `kube-apiserver.yaml`; inspect with `crictl logs`

Exam-day checklist

- Static Pods are run by the **kubelet** from `staticPodPath` (default `/etc/kubernetes/manifests`), no scheduler.
- Find the path via `grep staticPodPath /var/lib/kubelet/config.yaml`.
- The API shows a read-only **mirror Pod** named `<pod>-<nodename>`.
- To create/delete/edit: work with the **manifest file on the node**, not `kubectl`.
- The control plane itself is static Pods — edit their manifests to change flags; use `crictl` when the API is unreachable.

Node Maintenance: cordon, drain & uncordon

Nodes need downtime — a kernel patch, a kubelet/kubeadm upgrade, a hardware swap. Kubernetes gives you three verbs to take a node out of service **gracefully** so running workloads are rescheduled elsewhere instead of being killed: `cordon` (stop new Pods landing here), `drain` (evict what is already here), and `uncordon` (put the node back in the pool). This is a near-certain exam task and it is the front half of the cluster-upgrade workflow, so drill it until it is muscle memory.

The three verbs at a glance

Command	What it does	Effect on existing Pods
`k cordon <node>`	Marks node unschedulable	None — they keep running
`k drain <node>`	Cordons *and* evicts Pods	Evicted & rescheduled elsewhere
`k uncordon <node>`	Marks node schedulable again	None — new Pods can land again

Cordon — the "no new tenants" sign

Cordoning sets `spec.unschedulable: true` on the Node. The scheduler then skips it for new Pods, but nothing already running is touched. In `k get nodes` the STATUS column gains `SchedulingDisabled`.

```bash
k cordon node01
k get nodes
# NAME     STATUS                     ROLES    AGE   VERSION
# node01   Ready,SchedulingDisabled   <none>   40d   v1.30.2
```

> 📌 **NOTE** Cordon is purely a scheduling gate. It does **not** move or restart anything — use it when you want to stop attracting new work before you drain, or to quarantine a flaky node while you investigate.

Drain — evict the current tenants

`drain` first cordons the node, then evicts every eligible Pod using the Eviction API (which respects PodDisruptionBudgets). Controllers (Deployments, ReplicaSets, StatefulSets) then recreate those Pods on other schedulable nodes. A bare `k drain` will refuse to run because of a few Pod categories, so you almost always need flags.

```bash
k drain node01 \
  --ignore-daemonsets \        # DaemonSet Pods can't move — skip them
  --delete-emptydir-data \     # allow evicting Pods that use emptyDir (data is lost)
  --force                      # evict "unmanaged" Pods with no controller
```

Flag	Why drain needs it
`--ignore-daemonsets`	DaemonSet Pods are pinned to the node; drain errors out unless told to ignore them (it will not delete them — they stay).

`--delete-emptydir-data`	A Pod with an `emptyDir` holds local data that cannot be moved; drain refuses unless you accept the data loss.
`--force`	Evicts standalone Pods not backed by a controller (bare Pods, static Pods). Without a controller they will **not** be recreated — they just vanish.
`--grace-period`	Override the Pod's termination grace period (seconds).
`--timeout`	Give up the drain after e.g. `5m` instead of blocking forever.

⚠️ **GOTCHA** Static Pods (mirror Pods managed by the kubelet, e.g. the control-plane components under `/etc/kubernetes/manifests`) cannot be evicted by drain. Drain will warn and skip them; they only stop when you move or delete their manifest file.

What PodDisruptionBudgets do to a drain

Drain evicts through the **Eviction API**, which enforces any **PodDisruptionBudget** (PDB) covering the Pods. A PDB declares how much voluntary disruption an application can tolerate via `minAvailable` or `maxUnavailable`. If evicting the next Pod would violate the budget, the eviction is rejected and `drain` **blocks and retries** until the app's other replicas are healthy enough to allow it.

YAML
```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web-pdb
spec:
  minAvailable: 2            # never let fewer than 2 web Pods run at once
  selector:
    matchLabels:
      app: web
```

🎯 **EXAM TIP** If a drain hangs, suspect a PDB. A **misconfigured** PDB (e.g. `minAvailable` equal to the replica count, or a single-replica Deployment with `minAvailable: 1`) makes eviction mathematically impossible and the drain never completes. Check with `k get pdb -A` and `k describe pdb <name>` (look at `ALLOWED DISRUPTIONS`).

Uncordon — welcome tenants back

After the maintenance is done, `uncordon` clears `spec.unschedulable` so the scheduler can place new Pods on the node again. Note it does **not** pull back the Pods that were evicted — they stay where they were rescheduled. Rebalancing happens naturally over time (or via the descheduler).

BASH
```bash
k uncordon node01
k get node node01        # STATUS back to just "Ready"
```

The upgrade context — where this all fits

The canonical reason to drain is a `kubeadm` upgrade. You upgrade one node at a time, and you always drain first so the workloads are safely off the node while its kubelet restarts. The rhythm for a worker node is:

```bash
# --- from the control plane / your workstation ---
k drain node01 --ignore-daemonsets --delete-emptydir-data

# --- on node01 itself (ssh in) ---
apt-get update && apt-get install -y kubeadm=1.30.3-1.1
kubeadm upgrade node
apt-get install -y kubelet=1.30.3-1.1 kubectl=1.30.3-1.1
systemctl daemon-reload && systemctl restart kubelet

# --- back on the control plane ---
k uncordon node01
k get nodes              # confirm node01 Ready and on the new version
```

> 📌 **NOTE** On a control-plane node the sequence is similar but you run `kubeadm upgrade apply v1.30.3` (first control-plane node) or `kubeadm upgrade node` (additional ones), and you still drain/uncordon it.

Exam-day checklist

- Memorise the drain flag trio: `--ignore-daemonsets --delete-emptydir-data --force`. Most graders' nodes have DaemonSets, so `--ignore-daemonsets` is almost always required.
- **Drain hangs?** → check `k get pdb -A` for a budget with 0 ALLOWED DISRUPTIONS, and check for bare Pods needing `--force`.
- Cordon ≠ drain. Cordon only blocks *new* Pods; drain also removes the running ones.
- Always `uncordon` when finished — a node left `SchedulingDisabled` is a common way to lose points.
- `k drain` is idempotent-ish: re-running after fixing a PDB will continue the eviction.

The Troubleshooting Workflow

Troubleshooting is roughly **30% of the CKA** — the single largest domain — and it is where time pressure bites hardest. The winning strategy is not raw knowledge but a **systematic method**: always work from the outside in, one layer at a time, and let the evidence point you to the next command instead of guessing. This chapter gives you a decision tree across four layers — **Application**, **Control plane**, **Node**, and **Networking** — plus the exact command to run for each symptom.

> 🎯 **EXAM TIP** Golden rule: **describe before you guess, logs before you edit.** The `Events:` section of `kubectl describe` and the last lines of `kubectl logs` solve the majority of exam scenarios in seconds.

Layer A — Application (start here almost always)

Begin with the workload itself. The four-command opening sweep:

```bash
k get pods -o wide -A                 # STATUS + which node + restarts
k describe pod <pod>                   # scroll to Events: at the bottom
k logs <pod> [-c <container>]          # current container's stdout/stderr
k logs <pod> --previous                # logs of the CRASHED prior container
k exec -it <pod> -- sh                 # poke around inside a running container
```

The STATUS column is your first branch point. Map the symptom to its usual cause:

Pod status	Meaning	First move
`Pending`	Not scheduled to any node	`describe pod` → Events. Causes: no node with enough CPU/mem, unsatisfied nodeSelector/affinity, taints with no toleration, unbound PVC.
`ContainerCreating`	Scheduled but not started	Usually volume mount, Secret/ConfigMap missing, or image pull in progress. `describe pod` Events.
`ImagePullBackOff` / `ErrImagePull`	Can't fetch the image	Wrong image name/tag, private registry with no `imagePullSecret`, or no network. Check the exact image string in Events.
`CrashLoopBackOff`	Container starts then exits, repeatedly	`k logs <pod> --previous` — the app is dying. Bad command, missing config/env, failing liveness probe.
`OOMKilled`	Killed for exceeding memory limit	Seen in `describe` under Last State. Raise the memory `limits` or fix the leak.
`Error` / `Completed`	Container exited (non-zero / zero)	Check exit code and `logs --previous`. For Jobs, `Completed` is success.
`Init:0/1`	Stuck in an initContainer	`k logs <pod> -c <init-container>` — the init step is failing.
`Terminating` (stuck)	Deletion blocked	Finalizer or a node that is gone. Investigate before force-deleting.

> ⚠️ **GOTCHA** `CrashLoopBackOff` is not an error itself — it means "the container keeps crashing and I'm backing off before restarting it." The real error is always in `logs --previous`. Reaching for `describe` alone here wastes time.

Deeper application checks:

```bash
k get events -A --sort-by=.lastTimestamp   # cluster-wide timeline of what happened
k describe deploy <name>                    # replica math, rollout conditions
k get pod <pod> -o yaml | less              # inspect actual applied spec, probes, env
```

Layer B — Control plane

If the API server is answering (`kubectl` works) but cluster behaviour is broken — Pods not scheduling, controllers not acting, nodes not updating — inspect the control-plane components. In a kubeadm cluster these run as **static Pods** defined by manifests in `/etc/kubernetes/manifests/` on the control-plane node.

```bash
k get pods -n kube-system            # apiserver, etcd, controller-manager, scheduler
k describe pod -n kube-system kube-scheduler-controlplane
k logs -n kube-system kube-apiserver-controlplane
```

If `kubectl` itself fails ("connection refused"), the API server is down and you must go to the node and use the container runtime directly (see the crictl chapter):

```bash
# on the control-plane node
sudo crictl ps -a                    # list containers incl. exited ones
sudo crictl logs <container-id>      # apiserver/etcd logs when kubectl can't reach them
sudo journalctl -u kubelet -f        # is the kubelet even starting the static Pods?
ls /etc/kubernetes/manifests/        # the four static-Pod manifests live here
sudo cat /var/log/pods/< ... >/ ...  # kubelet-written container logs on disk
```

> ⚠️ **GOTCHA** A classic control-plane break is a **bad edit to a static Pod manifest** (wrong flag, bad indentation, wrong port). The kubelet keeps trying to restart the broken Pod. Fix the YAML in `/etc/kubernetes/manifests/` and the kubelet re-applies it automatically — no restart needed. Watch out for a wrong `--etcd-servers`, cert paths, or the apiserver `--secure-port`.

Layer C — Node

A node showing `NotReady` almost always means its **kubelet** is unhealthy or not reporting. Work from the node's perspective:

```bash
k get nodes                          # which node is NotReady?
k describe node <node>               # Conditions: MemoryPressure, DiskPressure, PIDPressure

# ssh to the node, then:
sudo systemctl status kubelet        # is it active (running)?
sudo journalctl -u kubelet -f        # the real error scrolls here
sudo systemctl restart kubelet       # after fixing config
```

```
df -h                              # DiskPressure? full disk stops the kubelet
top / free -m                      # CPU / memory exhaustion
```

NotReady cause	Clue / fix
kubelet stopped	`systemctl status kubelet` shows inactive/failed → `systemctl enable --now kubelet`
Bad kubelet config	journalctl shows a parse/flag error → fix `/var/lib/kubelet/config.yaml` or `/etc/kubernetes/kubelet.conf`
Container runtime down	journalctl: "container runtime not ready" → `systemctl status containerd`
Disk/memory pressure	`describe node` Conditions → free space (`df -h`) or reduce load
Expired certs / wrong CA	kubelet can't authenticate to apiserver → check kubeconfig & cert dates

Layer D — Networking

Symptoms here: a Service returns nothing, DNS names don't resolve, or Pods can't reach each other. Trace the chain Service → Endpoints → Pod, then DNS, then kube-proxy.

BASH

```
# 1. Does the Service have backends? Empty ENDPOINTS = selector mismatch
k get endpoints <svc>
k get svc <svc> -o wide
k describe svc <svc>               # compare Selector to the Pods' labels

# 2. Can you reach the Service from inside the cluster?
k run test --rm -it --image=busybox:1.28 -- sh
  wget -qO- <svc>.<ns>:<port>
  nslookup <svc>.<ns>.svc.cluster.local

# 3. DNS itself
k get pods -n kube-system -l k8s-app=kube-dns   # CoreDNS running?
k logs -n kube-system -l k8s-app=kube-dns

# 4. kube-proxy / CNI
k get pods -n kube-system -l k8s-app=kube-proxy
k get pods -n kube-system                       # is the CNI (Calico/Flannel) healthy?
```

🎯 **EXAM TIP** Empty `ENDPOINTS` on a Service is the most common networking bug on the exam: the Service `selector` does not match the Pods' `labels` , or the Pods aren't Ready. Fix the selector or the labels so they align, and endpoints appear.

📌 **NOTE** Use the pinned `busybox:1.28` for `nslookup` — newer busybox images have a broken `nslookup` that gives misleading DNS failures.

The decision tree in one breath

1. Does `kubectl` respond? **No** → Layer B/C: go to the node, use `crictl` + `journalctl -u kubelet`.
2. Is the Pod scheduled and running? **No** → Layer A: `describe pod` Events (Pending/ImagePull/etc.).
3. Running but crashing? → `logs --previous` (CrashLoopBackOff / OOMKilled).
4. App healthy but unreachable? → Layer D: `get endpoints`, DNS, kube-proxy.
5. Node `NotReady`? → Layer C: kubelet status, journalctl, disk.

Exam-day checklist

- Always read `Events:` in `describe` first — it names the cause outright.
- `logs --previous` for anything crash-looping; the live container has no useful logs.
- When `kubectl` is dead, the answer is on the node: `crictl` and `journalctl -u kubelet`.
- Empty Service endpoints = label/selector mismatch. Prove it with `k get endpoints`.
- Don't fix blind. One layer, one command, read the output, then act.

Node & Container Debugging: crictl, journalctl & systemd

Sometimes `kubectl` can't help you — the API server is down, the node is `NotReady`, or the very Pod you need to inspect is a static control-plane Pod that won't come up. When that happens you SSH to the node and debug at the layer beneath Kubernetes: the **container runtime** (containerd, via the CRI) with `crictl`, and the **kubelet** as a systemd service with `systemctl` and `journalctl`. This is core Troubleshooting material and shows up whenever a scenario says "the cluster is broken and kubectl doesn't respond."

> 📌 **NOTE** Modern kubeadm clusters use **containerd** as the runtime (Docker's dockershim was removed in 1.24). The CRI client is `crictl` — it talks to any CRI-compliant runtime, so the same commands work on CRI-O too.

crictl — kubectl's shape, one level down

`crictl` lists the actual containers and Pod sandboxes the runtime knows about, independent of the API server. Its subcommands deliberately mirror `kubectl` / `docker`, so it feels familiar. Run it with `sudo`.

```bash
sudo crictl ps                # running containers
sudo crictl ps -a             # ALL containers, including exited (crucial for crashes)
sudo crictl pods              # Pod sandboxes and their state
sudo crictl images            # images present on the node

sudo crictl logs <container-id>        # stdout/stderr of a container
sudo crictl logs --tail=50 <id>        # last 50 lines
sudo crictl inspect <container-id>     # full JSON: mounts, env, exit code, cmd
sudo crictl exec -it <container-id> sh # shell into a running container
sudo crictl stop <id> / rm <id>        # stop / remove a container
```

> 🎯 **EXAM TIP** Use `crictl ps -a` when a control-plane component keeps restarting: the container is `Exited`, so plain `ps` hides it. Grab the ID, then `crictl logs <id>` to see why the apiserver or etcd died.

> ⚠️ **GOTCHA** If `crictl` prints "connection refused" or a deprecation warning about the endpoint, set the runtime socket. On containerd: `sudo crictl --runtime-endpoint unix:///run/containerd/containerd.sock ps`, or persist it in `/etc/crictl.yaml`: `runtime-endpoint: unix:///run/containerd/containerd.sock`.

Container logs on disk: /var/log/pods

The kubelet writes every container's stdout/stderr to the node's filesystem. When even `crictl logs` is unavailable, read the files directly:

```bash
ls /var/log/pods/                      # <namespace>_<pod>_<uid>/<container>/
ls /var/log/pods/kube-system_kube-apiserver-controlplane_*/kube-apiserver/
sudo tail -f /var/log/pods/<...>/0.log
ls /var/log/containers/                # symlinks pointing into /var/log/pods
```

The kubelet as a systemd service

The kubelet is **not** a container — it is a systemd-managed process on every node. When a node is `NotReady`, the kubelet is your prime suspect. The workflow:

```bash
sudo systemctl status kubelet          # active (running)? or failed/inactive?
sudo journalctl -u kubelet             # full log
sudo journalctl -u kubelet -f          # follow live
sudo journalctl -u kubelet --since "5 min ago" -p err   # recent errors only

sudo systemctl restart kubelet         # apply config changes / recover
sudo systemctl enable --now kubelet    # start it and enable at boot
sudo systemctl daemon-reload           # after editing the systemd unit/drop-in
```

> 🎯 **EXAM TIP** `journalctl -u kubelet` is the highest-value command for a NotReady node. It states the cause in plain text: a bad flag, a missing cert, a runtime that isn't ready, or a config parse error.

Where the kubelet's config lives

The kubelet gets its settings from several files. Knowing the paths lets you fix a broken kubelet fast.

Path	What it holds
`/var/lib/kubelet/config.yaml`	Main kubelet config (cgroup driver, cluster DNS, static-pod path, etc.)
`/etc/kubernetes/kubelet.conf`	kubeconfig the kubelet uses to reach the API server
`/etc/systemd/system/kubelet.service.d/10-kubeadm.conf`	systemd drop-in with kubeadm-managed flags
`/var/lib/kubelet/kubeadm-flags.env`	Runtime flags kubeadm writes (e.g. the CRI socket)
`/etc/kubernetes/manifests/`	Static Pod manifests the kubelet watches (control-plane components)

> 📌 **NOTE** The `staticPodPath` in `config.yaml` defaults to `/etc/kubernetes/manifests`. Drop a Pod manifest there and the kubelet runs it with no API server involved — that's how the control plane bootstraps itself. Change that path (or the file) and `systemctl restart kubelet` to pick it up.

Checking the container runtime itself

If the kubelet log says "container runtime is down" or "failed to get sandbox," verify containerd:

```bash
sudo systemctl status containerd
sudo journalctl -u containerd -f
sudo systemctl restart containerd
sudo crictl info                       # runtime + CNI status as JSON
sudo crictl version                    # confirms crictl can reach the runtime
```

A worked recovery: NotReady node

```bash
k get nodes                              # node01 NotReady
# ssh node01
sudo systemctl status kubelet            # inactive (dead)
sudo journalctl -u kubelet -e            # "failed to load config …/config.yaml: yaml: …"
sudo vi /var/lib/kubelet/config.yaml     # fix the malformed YAML
sudo systemctl restart kubelet
sudo systemctl status kubelet            # active (running)
# back on control plane
k get nodes                              # node01 Ready
```

Exam-day checklist

- kubectl dead? Go to the node: `crictl ps -a` + `crictl logs` for control-plane containers; `journalctl -u kubelet` for the kubelet.
- Always `crictl ps -a` (with `-a`) — crashed containers are `Exited` and hidden otherwise.
- NotReady node → `systemctl status kubelet` then `journalctl -u kubelet`; the error is spelled out there.
- After editing kubelet config, `systemctl restart kubelet`; after editing the unit file, `systemctl daemon-reload` first.
- Container logs also live on disk at `/var/log/pods/` when everything else fails.
- Remember the runtime socket: `unix:///run/containerd/containerd.sock`.

Helm

Helm is the de-facto **package manager for Kubernetes**. Instead of hand-applying a dozen YAML files, you install a versioned, templated bundle called a **chart** and manage its lifecycle — upgrade, roll back, uninstall — as a single unit. As of the 2024 CKA refresh, Helm (alongside Kustomize) is explicitly in the curriculum under the "Application deployment" area, so you must be able to add repos and install/upgrade releases quickly on the command line.

The three core concepts

Term	What it is
Chart	A package: templated Kubernetes manifests + metadata (`Chart.yaml`) + default values (`values.yaml`).
Release	A specific installation of a chart into a cluster, with a name. Install the same chart twice = two releases.
Values	The configuration knobs that fill in the templates. Defaults come from `values.yaml` ; you override them at install time.
Repository	An HTTP server hosting packaged charts and an `index.yaml` catalogue.

> 📌 **NOTE** Helm 3 (current) is **Tiller-free** — there is no server-side component. The client renders templates locally and talks straight to the API server using your kubeconfig. Release state is stored as Secrets in the release's namespace.

Chart anatomy

```bash
mychart/
├── Chart.yaml          # name, version, appVersion, dependencies
├── values.yaml         # default configuration values
├── templates/          # Go-templated manifests (Deployment, Service, ...)
│   ├── deployment.yaml
│   ├── service.yaml
│   └── _helpers.tpl    # reusable template snippets
└── charts/             # bundled sub-chart dependencies
```

Working with repositories

```bash
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update                # refresh the local cache of all repos' index.yaml
helm repo list
helm search repo nginx          # find charts in added repos
helm search hub wordpress       # search the public Artifact Hub
```

Installing and inspecting releases

```bash
helm install web bitnami/nginx          # release "web" from chart bitnami/nginx
helm install web bitnami/nginx -n web-ns --create-namespace
```

```
helm list                               # releases in current namespace
helm list -A                            # all namespaces
helm status web                         # detailed state of release "web"
helm get values web                     # values in effect for the release
helm get manifest web                   # the actual rendered YAML applied
```

🎯 **EXAM TIP** Chart reference forms you'll see: `repo/chart` (from an added repo), a local path `./mychart`, or a packaged `.tgz`. Pin a version with `--version 15.4.4` for reproducibility.

Passing configuration: --set and -f

Two ways to override the chart's default values:

BASH

```
# inline overrides (dotted keys, comma-separated)
helm install web bitnami/nginx \
  --set replicaCount=3 \
  --set service.type=NodePort

# a values file (repeatable; later files win)
helm install web bitnami/nginx -f my-values.yaml
helm install web bitnami/nginx -f base.yaml -f prod.yaml
```

Precedence, lowest to highest: chart `values.yaml` → each `-f` file in order → `--set` flags. So `--set` always wins.

YAML

```
# my-values.yaml
replicaCount: 3
service:
  type: NodePort
image:
  tag: "1.27"
```

Upgrade, rollback, uninstall

BASH

```
helm upgrade web bitnami/nginx --set replicaCount=5     # change the release
helm upgrade --install web bitnami/nginx -f prod.yaml   # install if absent, else upgrade

helm history web                # revision list with status
helm rollback web 1             # revert to revision 1 (creates a new revision)

helm uninstall web              # remove the release and its resources
helm uninstall web --keep-history
```

📌 **NOTE** Every `install` / `upgrade` / `rollback` creates a new **revision**. `helm history` shows them; `helm rollback` to a prior number is your instant undo — a favourite exam-style task.

Rendering without installing: helm template

`helm template` runs the templating engine locally and prints the resulting manifests **without contacting the cluster**. Perfect for reviewing what a chart will do, or piping into `kubectl apply` in GitOps flows.

```bash
helm template web bitnami/nginx --set replicaCount=2   # print rendered YAML
helm template web ./mychart -f prod.yaml | k apply -f -
helm install web ./mychart --dry-run --debug           # render + validate server-side
```

> 🎯 **EXAM TIP** Under time pressure, `helm template ... | kubectl apply -f -` sidesteps release tracking entirely — handy when a task just wants the resources created, not managed. But `--dry-run --debug` is the better "what will happen?" check because it also runs server-side validation.

Why Helm is now in the CKA

Real clusters rarely run raw YAML — ingress controllers, metrics servers, CNIs and databases are almost all shipped as Helm charts. The exam added Helm because operating a cluster means installing and upgrading these packaged applications. Expect tasks like "add this repo and install chart X with these values" or "roll release Y back to its previous revision."

Exam-day checklist

- Sequence to remember: `helm repo add` → `helm repo update` → `helm install`. Forgetting `repo update` gives "chart not found."
- Values precedence: `--set` beats `-f` beats chart defaults.
- `helm upgrade --install` is the idempotent one-liner for "make it exist as configured."
- Instant undo: `helm history <rel>` then `helm rollback <rel> <rev>`.
- `helm list -A` to find a release when you don't know its namespace.
- `helm get manifest` shows exactly what was applied; `helm template` shows what would be applied offline.

Kustomize

Kustomize is Kubernetes' built-in, **template-free** way to customise YAML. Instead of parameterising manifests with a templating language (Helm's approach), you keep plain, valid Kubernetes YAML and layer declarative **overlays** on top of a shared **base**. It ships inside `kubectl` (`-k` / `kubectl kustomize`), so there is nothing to install — and, like Helm, it entered the CKA curriculum in the 2024 refresh under application deployment.

The kustomization.yaml file

Every Kustomize directory contains a `kustomization.yaml`. It lists the resources to include and the transformations to apply. Running Kustomize against that directory emits the final, combined YAML.

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:                # the raw manifests to include
  - deployment.yaml
  - service.yaml
namespace: dev            # set namespace on everything
namePrefix: dev-          # prefix every resource name
commonLabels:             # add labels to every resource + selectors
  team: platform
```

Bases and overlays

The core pattern: a **base** holds the common, environment-agnostic manifests; each **overlay** (dev, staging, prod) references the base and patches it with environment-specific changes. This eliminates copy-paste between environments.

```bash
app/
├── base/
│   ├── kustomization.yaml     # lists deployment.yaml, service.yaml
│   ├── deployment.yaml
│   └── service.yaml
└── overlays/
    ├── dev/
    │   └── kustomization.yaml # resources: [../../base] + dev patches
    └── prod/
        └── kustomization.yaml # resources: [../../base] + prod patches
```

```yaml
# overlays/prod/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base          # pull in everything from the base
namePrefix: prod-
commonLabels:
  env: prod
patches:
  - path: replicas-patch.yaml
    target:
```

```
      kind: Deployment
      name: web
```

> 📌 **NOTE** A base is just another directory with its own `kustomization.yaml`. Overlays reference it via `resources:` (the older `bases:` field is deprecated but still works). Bases can even be remote Git URLs.

Patches — modifying base resources

Overlays change base resources through patches. There are two mechanisms; both are set under the modern `patches:` field, and the legacy `patchesStrategicMerge` still appears in older material.

Strategic merge patch

You write a partial manifest; Kustomize merges it into the matching resource by name and kind. Intuitive for changing fields like replica count or image.

YAML
```yaml
# replicas-patch.yaml  (strategic merge — just the fields you change)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 5
```

YAML
```yaml
# kustomization.yaml — modern form
patches:
  - path: replicas-patch.yaml

# ... or the legacy field (same effect)
patchesStrategicMerge:
  - replicas-patch.yaml
```

JSON 6902 patch

For precise operations (add/replace/remove at an exact path) use an inline JSON patch with a `target` selector — ideal when a strategic merge is ambiguous.

YAML
```yaml
patches:
  - target:
      kind: Deployment
      name: web
    patch: |-
      - op: replace
        path: /spec/template/spec/containers/0/image
        value: nginx:1.27
```

Cross-cutting transformers

Field	Effect

`namePrefix` / `nameSuffix`	Prepend/append to every resource name (references are updated too).
`namespace`	Force all resources into one namespace.
`commonLabels`	Add labels to every resource **and** to selectors/templates — powerful but it also rewrites selectors.
`commonAnnotations`	Add annotations to every resource.
`images`	Override image name/tag without a patch: `images: [{name: nginx, newTag: "1.27"}]`.
`replicas`	Set replica counts by resource name.

⚠️ **GOTCHA** `commonLabels` injects the label into Deployment/Service **selectors** as well. Selectors are immutable, so adding a common label to an already deployed Deployment can break the apply. Prefer `labels:` with `includeSelectors: false`, or set common labels from the start.

Generators: ConfigMaps and Secrets

Kustomize can generate ConfigMaps/Secrets from literals or files, and — crucially — appends a **content hash** to the generated name. When the data changes the name changes, which forces Pods to roll (automatic config-driven redeploys).

YAML

```yaml
configMapGenerator:
  - name: app-config
    literals:
      - LOG_LEVEL=debug
    files:
      - app.properties

secretGenerator:
  - name: db-creds
    literals:
      - password=s3cr3t

generatorOptions:
  disableNameSuffixHash: true   # opt out of the hash if you need a stable name
```

Running Kustomize

BASH

```bash
# render only — print the combined YAML, no cluster contact
kubectl kustomize overlays/prod
kustomize build overlays/prod           # standalone binary, same output

# render AND apply in one step (note -k, not -f)
kubectl apply -k overlays/prod
kubectl apply -k .                      # kustomization.yaml in current dir

# preview a diff / delete
```

```
kubectl diff -k overlays/prod
kubectl delete -k overlays/prod
```

🎯 **EXAM TIP** The exam-critical distinction: `-f` applies a plain manifest file/dir; `-k` tells kubectl to run Kustomize against a directory containing a `kustomization.yaml`. Use `kubectl kustomize <dir>` first to eyeball the output, then `kubectl apply -k <dir>`.

📌 **NOTE** The `kubectl` built-in Kustomize can lag the standalone `kustomize` binary by a version, so a newest-syntax feature may work with `kustomize build` but not `kubectl -k`. On the exam, prefer the built-in and stick to stable fields.

Helm vs Kustomize in one line

Helm **templates** YAML with a Go engine and manages versioned releases; Kustomize **overlays** plain YAML with no templating and no release state. Different philosophies; the CKA expects basic fluency in both.

Exam-day checklist

- Apply with `kubectl apply -k <dir>` (dash-k), not `-f`.
- Preview with `kubectl kustomize <dir>` before applying.
- Base + overlay: overlays list `resources: [../../base]` then patch.
- Two patch styles: `patchesStrategicMerge` (partial manifest) and JSON 6902 (`op/path/value`) — both live under `patches:` in the modern form.
- Generators add a name hash by default; disable it with `disableNameSuffixHash: true` if a stable name is required.
- `commonLabels` also rewrites selectors — watch for immutable-selector errors on existing Deployments.

PART IV

30 Posts

Thirty bite-sized posts — each works as a LinkedIn post for people preparing, and as a spaced-revision card for you. One concept, one insight, one command to remember.

30 Posts for Revision & Sharing

Thirty bite-sized posts — each is both a LinkedIn card for fellow CKA candidates and a spaced-revision flashcard for me. One hook, one insight, one thing to remember. Read one a day and the whole syllabus revises itself.

Post 01 — The exam clock is your real opponent

The CKA isn't hard because of the concepts — it's hard because you have ~7 minutes per task. Hand-typing YAML from memory is how people run out of time. The move: generate a skeleton imperatively, then edit only what changes.

```bash
k run nginx --image=nginx --dry-run=client -o yaml > pod.yaml
k create deploy web --image=nginx --replicas=3 \
  --dry-run=client -o yaml > deploy.yaml
```

Remember: `--dry-run=client -o yaml` turns kubectl into a YAML generator — never start from a blank file.

#CKA #Kubernetes #kubectl #DevOps

Post 02 — The 30-second setup that saves 30 minutes

Before you touch a single question, paste these into your shell. `k` saves thousands of keystrokes; `do` and `now` make dry-run and fast-delete one-word habits.

```bash
alias k=kubectl
export do="--dry-run=client -o yaml"   # k run x --image=nginx $do
export now="--force --grace-period=0"   # k delete pod x $now
```

Remember: `k run web --image=nginx $do > web.yaml` — muscle memory beats typing.

#CKA #Kubernetes #Productivity #SRE

Post 03 — CrashLoopBackOff is a symptom, not a diagnosis

Everyone panics at CrashLoopBackOff. But it just means "container keeps exiting, so I'm backing off before restarting." The *cause* is in the logs — a bad command, missing config, a failing app. The status only tells you the container won't stay up.

```bash
k logs pod --previous     # logs from the crashed instance
k describe pod pod        # Last State + Exit Code + Events
```

Remember: Don't restart the pod — read `--previous` logs and the Exit Code. Exit 1 = app error, 137 = OOMKilled.

#CKA #Kubernetes #Troubleshooting #SRE

Post 04 — Init containers vs sidecars

Both live in the Pod, but they run at different times. **Init containers** run to completion, one after another, *before* app containers start — perfect for setup, migrations, waiting on a dependency. A **sidecar** (a native init container with `restartPolicy: Always` since 1.29) runs *alongside* the app for its whole life — logging, proxies, config reload.

```yaml
initContainers:
  - name: wait-db
    image: busybox
    command: ["sh","-c","until nc -z db 5432; do sleep 2; done"]
```

Remember: Init = runs then exits, blocks startup. Sidecar = runs forever, next to the app.

#CKA #Kubernetes #Pods #CloudNative

Post 05 — The three-layer chain you can undo

A Deployment doesn't manage Pods directly. **Deployment → ReplicaSet → Pods.** Each rollout creates a *new* ReplicaSet and scales the old one down — which is exactly why you can roll back in one command. The old ReplicaSet is still there at zero replicas, waiting.

```bash
k rollout status deploy/web
k rollout history deploy/web
k rollout undo deploy/web --to-revision=2
```

Remember: `rollout undo` works because Kubernetes keeps old ReplicaSets around — history is your safety net.

#CKA #Kubernetes #Deployments #DevOps

Post 06 — maxSurge and maxUnavailable control the blast radius

Rolling updates aren't magic — two numbers decide how aggressive they are. **maxSurge** = how many extra pods you can create above desired. **maxUnavailable** = how many you can drop below desired. Want zero downtime? maxUnavailable: 0. Want speed? Bump maxSurge.

```yaml
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1          # can be a count or %
    maxUnavailable: 0    # never go below desired capacity
```

Remember: maxUnavailable: 0 + maxSurge: 1 = safest zero-downtime rollout.

#CKA #Kubernetes #Deployments #SRE

Post 07 — Requests schedule, limits throttle

The single most confused pair in Kubernetes. **Requests** are what the scheduler uses to find a node with enough room — they reserve capacity. **Limits** are the ceiling the runtime enforces at runtime. Exceed a CPU limit and you get throttled; exceed a memory limit and you get OOMKilled (Exit 137).

```yaml
resources:
  requests: {cpu: "250m", memory: "128Mi"}   # used for scheduling
  limits:   {cpu: "500m", memory: "256Mi"}    # enforced at runtime
```

Remember: No node fits the *request* → Pod stays Pending. Exceed the memory *limit* → OOMKilled.

#CKA #Kubernetes #Scheduling #SRE

Post 08 — QoS decides who dies first

When a node runs out of memory, Kubernetes evicts pods in a strict order based on their QoS class — which you don't set directly, it's derived from your requests/limits. **Guaranteed** (requests == limits) is evicted last. **Burstable** (some set) is next. **BestEffort** (nothing set) is first out the door.

```bash
k get pod app -o jsonpath='{.status.qosClass}'   # Guaranteed | Burstable | BestEffort
```

Remember: Eviction order = BestEffort → Burstable → Guaranteed. Set requests == limits for your critical pods.

#CKA #Kubernetes #QoS #Reliability

Post 09 — Three probes, three jobs

They sound similar but answer different questions. **Liveness**: "is it stuck? restart it." **Readiness**: "can it take traffic? if not, pull it from the Service." **Startup**: "is it still booting? hold off the other two." Slow-starting apps need a startup probe or liveness will kill them mid-boot.

```yaml
livenessProbe:  {httpGet: {path: /healthz, port: 8080}, periodSeconds: 10}
readinessProbe: {httpGet: {path: /ready,   port: 8080}, periodSeconds: 5}
startupProbe:   {httpGet: {path: /healthz, port: 8080}, failureThreshold: 30}
```

Remember: Liveness restarts, readiness gates traffic, startup buys boot time.

#CKA #Kubernetes #Probes #Reliability

Post 10 — base64 is not encryption

ConfigMaps and Secrets are almost identical objects — the difference is intent, not security. Secret values are base64-*encoded*, which anyone can decode in one command. Real protection comes from RBAC and enabling encryption-at-rest in etcd.

```bash
k create secret generic db --from-literal=pass=s3cret
k get secret db -o jsonpath='{.data.pass}' | base64 -d   # anyone can read it
```

Remember: A Secret only keeps data out of your YAML — lock it down with RBAC + etcd encryption.

#CKA #Kubernetes #Secrets #Security

Post 11 — port vs targetPort vs nodePort

Three ports on one Service and everyone mixes them up. **port** = the Service's own port (what clients hit inside the cluster). **targetPort** = the container port traffic is forwarded to. **nodePort** = the port opened on every node for external access (30000–32767).

```yaml
ports:
  - port: 80          # the Service listens here
    targetPort: 8080  # forwards to container's 8080
    nodePort: 30080    # reachable on <nodeIP>:30080 (NodePort type)
```

Remember: Client → **port** → **targetPort** (container). **nodePort** is the external door.

#CKA #Kubernetes #Services #Networking

Post 12 — How a Service actually finds its pods

A Service is just a stable name and IP — the real work is done by **EndpointSlices** (the modern replacement for Endpoints). A controller watches pods matching the Service's selector and writes their IPs into slices. No ready pods matching the selector = empty endpoints = connection refused.

```bash
k get endpointslices -l kubernetes.io/service-name=web
k describe svc web        # check Selector vs pod labels
```

Remember: Service unreachable? Check EndpointSlices. Empty = selector mismatch or no ready pods.

#CKA #Kubernetes #Networking #Troubleshooting

Post 13 — Cluster DNS is predictable, so memorize it

Every Service gets a DNS name with a fixed shape. Same namespace? Just use the Service name. Cross-namespace? Add the namespace. The full form always works:

```bash
<service>.<namespace>.svc.cluster.local
# e.g. db.prod.svc.cluster.local
k run test --image=busybox --rm -it -- nslookup db.prod
```

Remember: `service.namespace.svc.cluster.local` — same-ns you can drop everything after the service name.

#CKA #Kubernetes #DNS #Networking

Post 14 — Default-deny is the NetworkPolicy foundation

Pods talk to everything by default. NetworkPolicies are additive allow-rules — but the powerful first step is a default-deny that closes the namespace, then you open only what's needed. An empty `podSelector` selects *all* pods.

```yaml
kind: NetworkPolicy
spec:
  podSelector: {}          # all pods in the namespace
  policyTypes: ["Ingress"]  # no ingress rules = deny all ingress
```

Remember: Empty podSelector + a policyType with no rules = default-deny. Then layer allow-policies on top.

#CKA #Kubernetes #NetworkPolicy #Security

Post 15 — An Ingress without a controller does nothing

Creating an Ingress resource is only half the job — it's just rules. Something has to *implement* them: an Ingress controller (nginx, Traefik, etc.). No controller running = your Ingress sits there with no address. Meanwhile the newer **Gateway API** is where the ecosystem is heading, with richer routing and role separation.

```bash
k get pods -n ingress-nginx      # is a controller actually running?
k get ingress web                # ADDRESS empty? no controller.
```

Remember: Ingress = rules; the controller = the engine. Gateway API is the successor.

#CKA #Kubernetes #Ingress #Networking

Post 16 — PVC binding rules (and the Pending trap)

A PVC binds to a PV only if **capacity ≥ request**, **access modes match**, and **storageClassName matches**. Miss any one and the PVC sits Pending. And with a topology-aware StorageClass, `WaitForFirstConsumer` intentionally delays binding until a pod is scheduled — so Pending is expected until then.

```bash
k describe pvc data     # Events tell you exactly why it's Pending
```

Remember: PVC Pending? Compare size, accessModes, storageClassName. If mode is WaitForFirstConsumer, it binds only once a pod uses it.

#CKA #Kubernetes #Storage #Troubleshooting

Post 17 — The "Released" PV won't come back on its own

Delete a PVC and the PV's reclaim policy decides what happens. **Delete** wipes the backing storage. **Retain** keeps your data — but marks the PV `Released`, and a Released PV will *never* auto-bind to a new claim. You have to manually clear its `claimRef` to make it Available again.

```bash
k patch pv pv-data -p '{"spec":{"claimRef":null}}'   # Released → Available
```

Remember: Retain protects data but strands the PV in Released — clear `spec.claimRef` to reuse it.

#CKA #Kubernetes #Storage #SRE

Post 18 — nodeSelector is a hard filter; affinity is a language

nodeSelector is the blunt tool: exact label match, all-or-nothing. **Node affinity** is the expressive version — it supports operators (In, NotIn, Exists), and both *required* (hard) and *preferred* (soft) rules. Use nodeSelector for simple pins, affinity when you need "prefer, but don't fail."

```yaml
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - {key: disktype, operator: In, values: ["ssd"]}
```

Remember: nodeSelector = exact-match hard rule. Affinity = operators + hard/soft preferences.

#CKA #Kubernetes #Scheduling #DevOps

Post 19 — Taints repel, tolerations permit

A **taint** on a node says "keep pods away unless they can tolerate me." A **toleration** on a pod says "I'm okay with that taint." Key point: a toleration doesn't *attract* a pod to the node — it just removes the repulsion. Use affinity if you actually want to pull pods in.

```bash
k taint nodes node1 gpu=true:NoSchedule
# pod tolerates it:
tolerations: [{key: gpu, operator: Equal, value: "true", effect: NoSchedule}]
```

Remember: Taint = node repels; toleration = pod is allowed (not attracted). Effects: NoSchedule, PreferNoSchedule, NoExecute.

#CKA #Kubernetes #Scheduling #SRE

Post 20 — DaemonSets: one pod per node, automatically

Need an agent on every node — a log shipper, a CNI, a metrics collector? That's a DaemonSet. It runs exactly one pod per matching node and automatically schedules onto new nodes as they join the cluster. No replica count to manage.

```bash
k get ds -A          # DESIRED matches node count
k get pods -o wide   # one pod per node
```

Remember: DaemonSet = one pod per node, self-adjusting as nodes come and go. It tolerates control-plane taints if you tell it to.

#CKA #Kubernetes #DaemonSet #Workloads

Post 21 — Jobs run to completion; CronJobs run on a schedule

Deployments keep pods running forever. A **Job** is the opposite — it runs a task until it succeeds, then stops. A **CronJob** is just a Job factory on a cron schedule. Watch the `restartPolicy`: Jobs use `Never` or `OnFailure`, never `Always`.

```bash
k create job pi --image=perl -- perl -Mbignum -e 'print bpi(200)'
k create cronjob report --image=busybox --schedule="0 * * * *" -- date
```

Remember: Job = run once to completion. CronJob = Job on a schedule. restartPolicy must be Never/OnFailure.

#CKA #Kubernetes #Jobs #Workloads

Post 22 — Role vs ClusterRole, and the question that ends debates

RBAC has two axes. **Role** is namespaced; **ClusterRole** is cluster-wide (and covers cluster-scoped resources like nodes). A RoleBinding grants within a namespace; a ClusterRoleBinding grants everywhere. When in doubt about what someone can do, just ask the API.

```bash
k auth can-i create pods --as=jane -n dev
k auth can-i '*' '*' --as=system:serviceaccount:dev:builder
```

Remember: Role/RoleBinding = namespaced. ClusterRole/ClusterRoleBinding = cluster-wide. `auth can-i` settles every argument.

#CKA #Kubernetes #RBAC #Security

Post 23 — Every pod has an identity, whether you set it or not

Pods authenticate to the API server as a **ServiceAccount**. Don't specify one? You get the namespace's `default` SA. A projected token is mounted into the pod, and RBAC bindings on that SA decide what the pod's code can do. This is how you give a controller pod exactly the permissions it needs — no more.

```bash
k create sa builder
k set serviceaccount deploy/ci builder   # bind SA to workload
```

Remember: No SA specified = the default SA. Grant least privilege by binding a Role to a dedicated ServiceAccount.

#CKA #Kubernetes #ServiceAccount #Security

Post 24 — Static pods answer to the kubelet, not the scheduler

Drop a manifest in `/etc/kubernetes/manifests` and the kubelet runs it directly — no API server, no scheduler involved. That's how the control plane bootstraps itself (apiserver, etcd, controller-manager all run as static pods). You see a read-only mirror pod in the API, but you can't delete it with kubectl — remove the file.

```bash
ls /etc/kubernetes/manifests/     # kube-apiserver.yaml, etcd.yaml ...
# edit or move a file to change/stop a static pod
```

Remember: Static pods live in `/etc/kubernetes/manifests` and are managed by editing files, not kubectl.

#CKA #Kubernetes #kubelet #ControlPlane

Post 25 — The etcd snapshot task you *will* see

etcd holds all cluster state, and backing it up is a near-guaranteed CKA task. Memorize the flags — the three certs plus the endpoint. Save is easy; restore points to a fresh data-dir, then you repoint the etcd static pod at it.

```bash
ETCDCTL_API=3 etcdctl snapshot save /opt/snap.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
```

Remember: save needs endpoint + cacert + cert + key. Restore with `--data-dir` to a new path, then update the etcd manifest.

#CKA #Kubernetes #etcd #Backup

Post 26 — Upgrade order: control plane first, one minor at a time

kubeadm upgrades follow a strict order and you cannot skip minor versions. Do the first control-plane node with `kubeadm upgrade apply`, then other control-plane nodes with `upgrade node`, then workers. On each node, upgrade the kubeadm binary first, then kubelet + kubectl. Drain before, uncordon after.

```bash
kubeadm upgrade plan
kubeadm upgrade apply v1.30.2      # first control-plane node
k drain node --ignore-daemonsets   # before touching kubelet
```

Remember: Control plane → workers, one minor at a time. Never jump 1.29 → 1.31.

#CKA #Kubernetes #kubeadm #Upgrades

Post 27 — cordon, drain, uncordon: the maintenance dance

Three verbs for taking a node out of service safely. **cordon** marks it unschedulable (existing pods stay). **drain** cordons *and* evicts the pods, respecting PodDisruptionBudgets. **uncordon** puts it back in rotation. You almost always need `--ignore-daemonsets`, and often `--delete-emptydir-data`.

```bash
k drain node1 --ignore-daemonsets --delete-emptydir-data
# ... do maintenance ...
k uncordon node1
```

Remember: cordon = stop new pods; drain = cordon + evict; uncordon = back to work.

#CKA #Kubernetes #NodeOps #SRE

Post 28 — The 4-layer troubleshooting method

When something's broken, don't poke randomly — descend the stack in order. **App** (logs, describe, events) → **Control plane** (are apiserver/scheduler/controller-manager healthy?) → **Node** (kubelet, disk, Ready status) → **Network** (Service endpoints, DNS, NetworkPolicy). Most CKA failures resolve in the first two layers.

```bash
k get events -A --sort-by=.lastTimestamp   # start here, always
k get pods -n kube-system                    # control-plane health
k get nodes                                   # node Ready?
```

Remember: App → Control plane → Node → Network. Start with events, sorted by time.

#CKA #Kubernetes #Troubleshooting #SRE

Post 29 — When kubectl can't help you, drop to the node

If the API server or kubelet is down, kubectl is useless — so you go directly to the node. **crictl** talks to the container runtime (list/inspect/log containers). **journalctl -u kubelet** shows why the kubelet won't start. This is the exam's "the node is NotReady, fix it" scenario.

```bash
crictl ps -a                      # containers the runtime sees
journalctl -u kubelet -f          # why kubelet is unhappy
systemctl status kubelet
```

Remember: No kubectl? crictl for containers, `journalctl -u kubelet` for the kubelet's own logs.

#CKA #Kubernetes #crictl #Troubleshooting

Post 30 — Exam day: killer.sh, the clock, and the flag button

The two free **killer.sh** sessions are harder than the real exam on purpose — treat them as your calibration. On the day: skip anything that stalls you and **flag it** for later, watch the weight of each question, set your context with `kubectl config use-context` every single task, and verify your work. Time management beats knowing every answer.

```bash
k config use-context <ctx>   # FIRST thing, every question
```

Remember: Do killer.sh twice, flag hard questions, switch context first, and never let one task eat your clock.

#CKA #Kubernetes #ExamPrep #Certification